

C++ DSA

Interview Handbook

Complete FAANG / MAANG Preparation Guide
From Beginner Syntax to Advanced DSA Mastery

- FAANG / MAANG (Google · Meta · Amazon · Apple · Netflix · Microsoft)
- Big Tech (Uber · Airbnb · Atlassian · Siemens)
- Global Banks (Morgan Stanley · Wells Fargo · Citi · HSBC)

Language	Timeline	Level
C++17	1 Month	Beginner → Advanced

Table of Contents

Section 1 C++ Fundamentals

Syntax · Types · OOP · Templates · Lambdas

Section 2 STL Deep Dive

vector · map · unordered_map · set · priority_queue · algorithms

Section 3 DSA Foundations

Complexity · Recursion · Sorting · Searching · Divide & Conquer

Section 4 Complete DSA Pattern System

15 patterns: Sliding Window · Two Pointers · DP · Backtracking · Graphs + more

Section 5 Competitive Programming Optimization

Fast I/O · Memory · STL tricks · Bitset · Mathematical optimizations

Section 6 Interview Preparation

FAANG mindset · 7-step framework · Communication · Company insights

Section 7 1-Month Roadmap

Daily plan · LeetCode Top 75 · Mock schedule · Progress tracker

C++ DSA Interview Handbook

Complete FAANG/MAANG Preparation Guide

***Language:** C++ | **Level:** Beginner to Advanced | **Target:** FAANG, MAANG, Big Tech, Global Banks, Product Companies*

Who This Is For

This handbook is designed for engineers with strong backend experience (MERN, AWS, DevOps) who need to master **Data Structures & Algorithms in C++** for top-tier technical interviews at:

- **FAANG/MAANG:** Google, Meta, Amazon, Apple, Netflix, Microsoft
- **Big Tech:** Uber, Airbnb, Atlassian, Siemens
- **Global Banks:** Morgan Stanley, Wells Fargo, Citi, HSBC

Handbook Structure

Section	Topic	Focus
<i><i><u>Section 1</u></i></i>	C++ Fundamentals	Syntax, OOP, Templates, Memory
<i><i><u>Section 2</u></i></i>	STL Deep Dive	All containers, algorithms, complexity
<i><i><u>Section 3</u></i></i>	DSA Foundations	Complexity, Recursion, Sorting, Searching
<i><i><u>Section 4</u></i></i>	Complete DSA Pattern System	15 major patterns with templates
<i><i><u>Section 5</u></i></i>	Competitive Programming	Fast I/O, Optimizations, Tricks
<i><i><u>Section 6</u></i></i>	Interview Preparation	Mindset, Communication, Strategy
<i><i><u>Section 7</u></i></i>	1-Month Roadmap	Daily plan, LeetCode roadmap

Quick Reference Cheat Sheets

STL Time Complexity Summary

Container	Access	Search	Insert	Delete
<i>vector</i>	O(1)	O(n)	O(1) amortized	O(n)
<i>array</i>	O(1)	O(n)	N/A	N/A

Container	Access	Search	Insert	Delete
<code>deque</code>	$O(1)$	$O(n)$	$O(1)$ both ends	$O(1)$ both ends
<code>list</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>map</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
<code>unordered_map</code>	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
<code>set</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
<code>unordered_set</code>	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
<code>priority_queue</code>	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$
<code>stack</code>	$O(1)$ top	$O(n)$	$O(1)$	$O(1)$
<code>queue</code>	$O(1)$ front	$O(n)$	$O(1)$	$O(1)$

Pattern Recognition Quick Guide

Problem Signal	Pattern
Subarray/substring of size k	Sliding Window
Sorted array, find pair/triplet	Two Pointers
Search in sorted/rotated array	Binary Search
Subarray sum, range queries	Prefix Sum
Frequency count, duplicates	HashMap/HashSet
Matching brackets, next greater	Stack
Tree traversal, path problems	Tree DFS/BFS
Shortest path, connected components	Graph BFS/DFS
Optimal substructure, overlapping subproblems	DP

Problem Signal	Pattern
Generate all combinations/permutations	Backtracking
K largest/smallest, median	Heap
Overlapping intervals	Intervals
Local optimal = Global optimal	Greedy
Prefix search, autocomplete	Trie
XOR, powers of 2	Bit Manipulation

Essential C++ Headers for Interviews

```
#include <bits/stdc++.h> // Includes everything – use in interviews
using namespace std;
```

```
// Or individually:
#include <iostream>;
#include <vector>;
#include <string>;
#include <map>;
#include <unordered_map>;
#include <set>;
#include <unordered_set>;
#include <queue>;
#include <stack>;
#include <algorithm>;
#include <numeric>;
#include <climits>;
#include <cmath>;
```

Complexity Analysis Quick Reference

Notation	Name	Example
$O(1)$	Constant	Array access, HashMap lookup
$O(\log n)$	Logarithmic	Binary search, BST operations
$O(n)$	Linear	Linear scan, Single loop
$O(n \log n)$	Linearithmic	Merge sort, Heap sort
$O(n^2)$	Quadratic	Nested loops, Bubble sort
$O(2^n)$	Exponential	Recursive Fibonacci, Subsets
$O(n!)$	Factorial	Permutations

Interview Communication Template

1. UNDERSTAND: Restate problem, clarify constraints
2. EXAMPLES: Walk through 2-3 examples including edge cases
3. BRUTE FORCE: State the naive approach + complexity
4. OPTIMIZE: Identify bottleneck, apply pattern
5. CODE: Write clean, commented code
6. TEST: Dry run with examples, test edge cases
7. COMPLEXITY: State time and space complexity

Section 1: C++ Fundamentals

From Syntax to Advanced OOP — Interview-Ready C++

*****Goal:**** Get fully comfortable writing C++ in interviews. Coming from JavaScript/MERN, this section bridges the gap.*

Table of Contents

1. Basic Syntax & Structure
2. Variables & Data Types
3. Operators
4. Conditions
5. Loops
6. Functions
7. Arrays
8. Strings
9. References & Pointers
10. Memory Basics
11. Structs
12. Classes & Objects
13. Constructors & Destructors
14. OOP Concepts
15. Inheritance
16. Polymorphism
17. Templates
18. Exception Handling
19. Lambda Functions

1. Basic Syntax & Structure

```
#include <iostream>;          // Include standard I/O library
#include <bits/stdc++.h>;      // Include ALL headers (use in interviews)
using namespace std;          // Avoid std:: prefix everywhere

int main() {
    // Entry point of every C++ program
    cout <<< "Hello, FAANG!" <<< endl; // Output
    cin >>> variable;                // Input
    return 0;
}
```

Key Differences from JavaScript

Feature	JavaScript	C++
Typing	Dynamic	Static (must declare type)
Compilation	Interpreted (JIT)	Compiled

Feature	JavaScript	C++
Memory	Garbage collected	Manual (RAII/smart ptrs)
Entry point	None (or module)	int main()
Semicolons	Optional	Required
Null	nullundefined	nullptrNULL

2. Variables & Data Types

```
// Integer types
int a = 42; // 32-bit, -2B to 2B
long long b = 1e18; // 64-bit – USE THIS for large numbers
short c = 100; // 16-bit
unsigned int d = 4294967295; // Only positive

// Floating point
float f = 3.14f; // 32-bit (avoid in interviews – precision issues)
double g = 3.14159265358979; // 64-bit – USE THIS
long double h = 3.14L; // 80/128-bit

// Characters & Boolean
char ch = 'A'; // Single character, 1 byte
bool flag = true; // true/false

// String
string s = "Hello"; // std::string (preferred)

// Auto (type deduction – very useful in interviews)
auto x = 42; // int
auto y = 3.14; // double
auto z = "hello"; // const char*
auto w = string("hello"); // string

// Constants
const int MAX = 1e9 + 7; // Cannot be changed
constexpr int SIZE = 100; // Compile-time constant
```

Important Constants for Interviews

```
#include <climits>;

INT_MAX = 2147483647 // 2^31 - 1
INT_MIN = -2147483648 // -2^31
LLONG_MAX = 9223372036854775807 // 2^63 - 1
LLONG_MIN = -9223372036854775808
DBL_MAX = 1.79769e+308

// Common interview values
const int MOD = 1e9 + 7; // Modular arithmetic
const int INF = 1e9; // Infinity approximation
const long long LINF = 1e18; // Long long infinity
```

Type Casting

```
int a = 5, b = 2;
```



```
double result = (double)a / b;           // C-style cast: 2.5
double result2 = static_cast<double>(a) / b; // C++ style (preferred)

// Implicit conversion pitfall
int x = 1000000;
int y = 1000000;
long long z = (long long)x * y; // MUST cast BEFORE multiplication!
// long long z = x * y; // WRONG: overflow happens before assignment
```

3. Operators

```
// Arithmetic
+ - * / %           // Standard arithmetic
a = b = c = 0;      // Chained assignment

// Comparison
== != < > <= >=

// Logical
&& || !             // AND, OR, NOT

// Bitwise (critical for competitive programming)
& // AND
| // OR
^ // XOR
~ // NOT (complement)
<< // Left shift (multiply by 2^n)
>> // Right shift (divide by 2^n)

// Examples
5 & 3 = 1 // 101 & 011 = 001
5 | 3 = 7 // 101 | 011 = 111
5 ^ 3 = 6 // 101 ^ 011 = 110
~5 = -6 // Flip all bits
5 << 1 = 10 // 5 * 2
5 >> 1 = 2 // 5 / 2

// Increment/Decrement
i++ // Post-increment (use value, then increment)
++i // Pre-increment (increment, then use value) – FASTER in loops
i-- // Post-decrement
--i // Pre-decrement

// Compound assignment
a += b; a -= b; a *= b; a /= b; a %= b;
a &= b; a |= b; a ^= b; a <<= 1; a >>= 1;

// Ternary
int max_val = (a > b) ? a : b;
```

4. Conditions

```
// if-else
if (n == 0) {
    cout << "zero";
} else if (n > 0) {
    cout << "positive";
} else {
    cout << "negative";
}

// switch (for discrete values)
```

```

switch (n) {
    case 1:
        cout <<< "one";
        break;
    case 2:
        cout <<< "two";
        break;
    default:
        cout <<< "other";
}

// Short-circuit evaluation
if (ptr != nullptr && ptr->val > 0) { // Safe: checks ptr first
    // ...
}

// Conditional with initialization (C++17)
if (auto it = map.find(key); it != map.end()) {
    // Use it->second
}

```

5. Loops

```

// for loop
for (int i = 0; i < n; i++) {
    // ...
}

// Range-based for (C++11) – very clean
vector<int> v = {1, 2, 3, 4, 5};
for (int x : v) cout << x << " "; // Read only
for (int& x : v) x *= 2;           // Modify in place
for (auto& [key, val] : myMap) {   // Structured binding C++17
    cout << key << ": " << val << "\n";
}

// while loop
while (condition) { /* ... */ }

// do-while (executes at least once)
do { /* ... */ } while (condition);

// Loop control
break; // Exit loop immediately
continue; // Skip to next iteration

// Nested loop with label-like break
bool found = false;
for (int i = 0; i < n && !found; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == target) {
            found = true;
            break;
        }
    }
}

```

6. Functions

```

// Basic function
int add(int a, int b) {
    return a + b;
}

```

```

}

// Pass by value (copy) – changes don't affect original
void modifyValue(int x) { x = 100; }

// Pass by reference – changes affect original (use in interviews!)
void modifyRef(int&x) { x = 100; }

// Pass by const reference – efficient read, no copy
void print(const string&s) { cout << s; }
void processVector(const vector&int&v) { /* read-only */ }

// Default parameters
int power(int base, int exp = 2) { /* ... */ }

// Function overloading
int max(int a, int b) { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }

// Inline functions (hint to compiler for small functions)
inline int square(int x) { return x * x; }

// Returning multiple values
pair<int, int> minMax(vector<int>&v) {
    return {*min_element(v.begin(), v.end()),
            *max_element(v.begin(), v.end())};
}

// auto return type (C++14)
auto divide(double a, double b) {
    return a / b;
}

// Recursive function
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}

```

7. Arrays

```

// C-style arrays (avoid in modern C++)
int arr[5] = {1, 2, 3, 4, 5};
int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};

// std::array (fixed size, use over C-style)
#include <array>
array<int, 5> arr = {1, 2, 3, 4, 5};
arr.size();    // 5
arr.front();   // 1
arr.back();    // 5

// std::vector (dynamic array – USE THIS in interviews)
vector<int> v; // Empty
vector<int> v(5); // Size 5, default 0
vector<int> v(5, -1); // Size 5, all -1
vector<int> v = {1, 2, 3, 4, 5}; // Initializer list
vector<vector<int>> grid(n, vector<int>(m, 0)); // 2D grid

// Vector operations
v.push_back(6); // Add to end: O(1) amortized
v.pop_back();   // Remove from end: O(1)
v.size();       // Number of elements
v.empty();      // Check if empty

```

```

v[i];                // Access: O(1)
v.at(i);             // Access with bounds check: O(1)
v.front();           // First element
v.back();            // Last element
v.insert(v.begin()+2, 99); // Insert at index: O(n)
v.erase(v.begin()+2); // Erase at index: O(n)
v.clear();           // Remove all elements
v.resize(10);        // Resize
v.reserve(100);      // Reserve capacity (performance!)

// Sorting
sort(v.begin(), v.end()); // Ascending
sort(v.begin(), v.end(), greater<int>()); // Descending
sort(v.begin(), v.end(), [](int a, int b){ return a > b; }); // Custom

// Searching
auto it = find(v.begin(), v.end(), target);
bool found = binary_search(v.begin(), v.end(), target); // Sorted required
auto lb = lower_bound(v.begin(), v.end(), x); // First >= x
auto ub = upper_bound(v.begin(), v.end(), x); // First > x

```

8. Strings

```

#include <string>
string s = "Hello, World!";

// Length
s.length(); // or s.size()

// Access
s[0];       // 'H'
s.front();  // 'H'
s.back();   // '!'

// Substring
s.substr(7, 5); // "World" (start, length)

// Find
s.find("World"); // Returns index or string::npos
s.find("xyz") == string::npos; // Not found

// Modify
s += " Extra"; // Concatenation
s.append(" More"); // Same as +=
s.replace(0, 5, "Hi"); // Replace "Hello" with "Hi"
s.erase(0, 7); // Erase from start
s.insert(0, "Prefix "); // Insert at position

// Conversion
to_string(42); // int → string
stoi("42"); // string → int
stoll("1234567890"); // string → long long
stod("3.14"); // string → double

// Character operations
#include <cctype>
isalpha('A'); // true
isdigit('5'); // true
isalnum('a'); // true
isupper('A'); // true
islower('a'); // true
toupper('a'); // 'A'
tolower('A'); // 'a'

// String comparison

```

```

s1 == s2;          // Equality
s1 < s2;           // Lexicographic

// Split string (no built-in, use stringstream)
#include <sstream>
string line = "hello world foo";
stringstream ss(line);
string word;
vector<string> words;
while (ss >> word) words.push_back(word);

// Iterate string
for (char c : s) { /* ... */ }
for (int i = 0; i < s.size(); i++) { /* s[i] */ }

// Reverse string
reverse(s.begin(), s.end());

// Sort string characters
sort(s.begin(), s.end());

// String to char array
const char* cstr = s.c_str();

```

9. References & Pointers

References

```

int x = 10;
int& ref = x;    // ref is an alias for x

ref = 20;        // x is now 20
cout << x;       // Prints 20

// Reference in functions – avoids expensive copy
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// Const reference – read-only, no copy
void display(const vector<int>& v) {
    for (const int& x : v) cout << x;
}

// Rules:
// - Must be initialized when declared
// - Cannot be re-assigned to another variable
// - Cannot be null

```

Pointers

```

int x = 10;
int* ptr = &x;    // ptr holds address of x

*ptr = 20;        // Dereference: change x to 20
cout << *ptr;     // 20
cout << ptr;      // Memory address (e.g., 0x7fff...)

// Null pointer
int* nullPtr = nullptr; // C++ style (use over NULL)

// Pointer arithmetic

```

```

int arr[] = {1, 2, 3, 4, 5};
int* p = arr;
p++;           // Point to next element
*(p+2);        // Element 2 positions ahead

// Pointer to pointer
int** pp = &p;
**pp = 30;

// Function pointer
int (*funcPtr)(int, int) = &add;
funcPtr(3, 4); // Calls add(3,4)

// Dangling pointer – AVOID
int* badPtr;
{
    int local = 5;
    badPtr = &local;
} // local destroyed – badPtr is now dangling!

```

Smart Pointers (Modern C++)

```

#include <memory>

// unique_ptr – sole ownership, auto-deleted
unique_ptr<int> p1 = make_unique<int>(42);
cout << *p1;

// shared_ptr – shared ownership, ref-counted
shared_ptr<int> p2 = make_shared<int>(42);
shared_ptr<int> p3 = p2; // Both point to same memory

// weak_ptr – non-owning reference (breaks circular refs)
weak_ptr<int> wp = p2;

```

10. Memory Basics

```

// Stack memory – automatic, fast, limited size
int arr[1000]; // Stack allocated

// Heap memory – manual, large, must be freed
int* p = new int(42); // Allocate single int
int* arr = new int[100]; // Allocate array
delete p; // Free single
delete[] arr; // Free array

// RAII – Resource Acquisition Is Initialization
// Use smart pointers, vectors, strings – they manage memory automatically

// Memory layout
// Stack: local variables, function calls
// Heap: dynamic allocation (new/delete)
// Text: compiled code
// Data: global/static variables

// Common memory errors (AVOID)
// 1. Memory leak: allocate but forget to delete
// 2. Double free: delete same memory twice
// 3. Use after free: use pointer after deleting
// 4. Buffer overflow: write past array bounds
// 5. Stack overflow: too deep recursion

```

11. Structs

```
struct Point {
    int x, y;
};

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};

// Usage
Point p = {3, 4};
Point q;
q.x = 1;
q.y = 2;

// Struct with methods (essentially a class)
struct Rectangle {
    int width, height;
    int area() { return width * height; }
    int perimeter() { return 2 * (width + height); }
};

// Comparison operator for sort
struct Interval {
    int start, end;
    bool operator<(const Interval& other) const {
        return start < other.start;
    }
};
```

12. Classes & Objects

```
class Animal {
private:          // Only accessible within class
    string name;
    int age;

protected:     // Accessible in class + derived classes
    string type;

public:          // Accessible everywhere
    // Constructor
    Animal(string n, int a) : name(n), age(a) {}

    // Getters
    string getName() const { return name; }
    int getAge() const { return age; }

    // Setters
    void setName(const string& n) { name = n; }

    // Methods
    void display() const {
        cout <<< name <<< " (Age: " <<< age <<< ")\n";
    }
};
```

```

    }

    // Static member (shared across all instances)
    static int count;
    static int getCount() { return count; }
};

int Animal::count = 0; // Define static outside class

// Create objects
Animal a("Dog", 5);
Animal* ptr = new Animal("Cat", 3); // Heap
ptr->display(); // Arrow for pointer
a.display(); // Dot for object
delete ptr;

```

13. Constructors & Destructors

```

class MyClass {
    int* data;
    int size;

public:
    // Default constructor
    MyClass() : data(nullptr), size(0) {}

    // Parameterized constructor
    MyClass(int s) : size(s) {
        data = new int[size];
    }

    // Copy constructor (deep copy)
    MyClass(const MyClass&& other) : size(other.size) {
        data = new int[size];
        copy(other.data, other.data + size, data);
    }

    // Move constructor (C++11) – efficient transfer
    MyClass(MyClass&& other) noexcept : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }

    // Copy assignment operator
    MyClass& operator=(const MyClass& other) {
        if (this != &other) { // Self-assignment check
            delete[] data;
            size = other.size;
            data = new int[size];
            copy(other.data, other.data + size, data);
        }
        return *this;
    }

    // Destructor (called automatically when object goes out of scope)
    ~MyClass() {
        delete[] data;
    }

    // Initializer list (preferred – more efficient)
    MyClass(int s, int val) : size(s), data(new int[s]) {
        fill(data, data + size, val);
    }
};

```


14. OOP Concepts

Encapsulation

```
class BankAccount {
private:
    double balance;    // Hidden from outside

public:
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    double getBalance() const { return balance; }
};
```

Abstraction

```
class Shape {
public:
    virtual double area() = 0;    // Pure virtual – must be overridden
    virtual ~Shape() {}
};
```

Operator Overloading

```
class Vector2D {
public:
    int x, y;
    Vector2D(int x, int y) : x(x), y(y) {}

    Vector2D operator+(const Vector2D& other) const {
        return {x + other.x, y + other.y};
    }

    bool operator==(const Vector2D& other) const {
        return x == other.x && y == other.y;
    }

    // For use in map/set
    bool operator<(const Vector2D& other) const {
        return x < other.x || (x == other.x && y < other.y);
    }

    // Output stream operator
    friend ostream& operator<<(ostream& os, const Vector2D& v) {
        os << "(" << v.x << ", " << v.y << ")";
        return os;
    }
};
```

15. Inheritance

```
class Animal {
protected:
    string name;
public:
    Animal(string n) : name(n) {}
    virtual void speak() { cout << name << " makes a sound\n"; }
    virtual ~Animal() {} // Always virtual destructor in base class!
};
```

```

class Dog : public Animal {
public:
    Dog(string n) : Animal(n) {}
    void speak() override { cout <<< name <<< " barks\n"; }
    void fetch() { cout <<< name <<< " fetches!\n"; }
};

class Cat : public Animal {
public:
    Cat(string n) : Animal(n) {}
    void speak() override { cout <<< name <<< " meows\n"; }
};

// Multiple inheritance
class FlyingFish : public Fish, public Bird {
    // ...
};

```

16. Polymorphism

```

// Runtime polymorphism (virtual functions)
Animal* animals[] = { new Dog("Rex"), new Cat("Whiskers") };
for (Animal* a : animals) {
    a->speak(); // Calls correct version at runtime
}

// Compile-time polymorphism (templates/overloading)
template<typename T>
T maximum(T a, T b) { return a > b ? a : b; }

// Pure virtual (Abstract class)
class Shape {
public:
    virtual double area() = 0;
    virtual double perimeter() = 0;
    virtual ~Shape() {}
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override { return 3.14159 * r * r; }
    double perimeter() override { return 2 * 3.14159 * r; }
};

```

17. Templates

```

// Function template
template<typename T>
T add(T a, T b) { return a + b; }

add(1, 2);           // int
add(1.5, 2.5);       // double

// Multiple type parameters
template<typename T, typename U>
auto multiply(T a, U b) { return a * b; }

// Class template
template<typename T>
class Stack {

```

```

        vector<T> data;
    public:
        void push(T val) { data.push_back(val); }
        T pop() {
            T top = data.back();
            data.pop_back();
            return top;
        }
        bool empty() { return data.empty(); }
};

Stack<int> intStack;
Stack<string> strStack;

// Template specialization
template<>
string add<string>(string a, string b) {
    return a + " " + b; // Custom behavior for strings
}

// Non-type template parameters
template<int N>
class FixedArray {
    int data[N];
public:
    int size() { return N; }
};

FixedArray<10> arr;

```

18. Exception Handling

```

#include <stdexcept>

// Basic try-catch
try {
    int result = divide(10, 0);
} catch (const exception& e) {
    cout <<< "Error: " <<< e.what() <<< "\n";
}

// Throw custom exception
void divide(int a, int b) {
    if (b == 0) throw runtime_error("Division by zero");
    return a / b;
}

// Multiple catch blocks
try {
    // risky code
} catch (const invalid_argument& e) {
    // Handle invalid argument
} catch (const out_of_range& e) {
    // Handle out of range
} catch (...) {
    // Catch everything else
}

// Exception hierarchy
// std::exception
//   ├── logic_error
//   ├── invalid_argument
//   ├── out_of_range
//   └── domain_error

```

```
// ■ ■ ■ runtime_error
// ■ ■ ■ overflow_error
// ■ ■ ■ underflow_error

// noexcept (promise not to throw)
int safeAdd(int a, int b) noexcept { return a + b; }
```

19. Lambda Functions

```
// Basic lambda
auto square = [](int x) { return x * x; };
cout <<< square(5); // 25

// Lambda with capture
int base = 10;
auto addBase = [base](int x) { return base + x; }; // Capture by value
auto addBaseRef = [&base](int x) { return base + x; }; // Capture by ref
auto captureAll = [=](int x) { return base + x; }; // Capture all by value
auto captureAllRef = [&](int x) { return base + x; }; // Capture all by ref

// Lambda as function argument (sort, transform, etc.)
vector<int> v = {5, 2, 8, 1, 9, 3};
sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

// Lambda with STL algorithms
vector<int> v = {1, 2, 3, 4, 5, 6};
auto it = find_if(v.begin(), v.end(), [](int x) { return x > 3; });

// Lambda in priority queue (custom comparator)
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.second > b.second; // Min-heap by second element
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);

// Mutable lambda (can modify captured copies)
int count = 0;
auto increment = [count]() mutable { return ++count; };
```

Interview-Ready C++ Patterns

Fast I/O Setup (Always put at top of main)

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    // your code
}
```

Common Interview Boilerplate

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef vector<int> vi;
typedef pair<int,int> pii;
typedef vector<pair<int,int>> vpii;

const int MOD = 1e9 + 7;
const int INF = 1e9;
```


Section 2: STL Deep Dive

Complete Standard Template Library for FAANG Interviews

***Philosophy:** Know every container's internal implementation, time complexity, and interview use case. The STL is your Swiss Army knife in C++ interviews.*

Table of Contents

1. `<vector>`
2. `<array>`
3. `<deque>`
4. `<list>`
5. `<map>`
6. `<unordered_map>`
7. `<set>`
8. `<unordered_set>`
9. `<multiset>`
10. `<multimap>`
11. `<priority_queue>`
12. `<stack>`
13. `<queue>`
14. `<pair & tuple>`
15. `<Algorithms Library>`
16. `<STL Optimization Tricks>`

1. vector

Internal Implementation

A **dynamic array** backed by a contiguous block of memory. When capacity is exceeded, it allocates $\sim 2x$ the current capacity and copies all elements (amortized $O(1)$ `push_back`).

Internal: `[1][2][3][4][_][_][_]`
 ^{size=4} ^{capacity=8}

Complexity Table

Operation	Complexity	Notes
Access [i]	$O(1)$	Direct indexing
push_back	$O(1)$ amortized	$O(n)$ on reallocation
pop_back	$O(1)$	
insert at front	$O(n)$	Shifts all elements

Operation	Complexity	Notes
insert at position	$O(n)$	Shifts elements
erase	$O(n)$	Shifts elements
find	$O(n)$	Linear scan
sort	$O(n \log n)$	
size/empty	$O(1)$	

Complete API Reference

```
#include <vector>
using namespace std;

// Construction
vector<int> v1; // Empty
vector<int> v2(5); // 5 zeros
vector<int> v3(5, 7); // {7,7,7,7,7}
vector<int> v4 = {1,2,3,4,5}; // Initializer list
vector<int> v5(v4.begin(), v4.begin()+3); // From iterator range
vector<vector<int>> grid(n, vector<int>(m, 0)); // 2D

// Size & Capacity
v.size(); // Number of elements
v.capacity(); // Allocated space
v.empty(); // true if size == 0
v.max_size(); // Maximum possible elements
v.resize(10); // Resize (fills new with 0)
v.resize(10, -1); // Resize, fill new with -1
v.reserve(100); // Reserve capacity (avoids reallocations)
v.shrink_to_fit(); // Release excess capacity

// Element access
v[i]; // No bounds check – FASTER
v.at(i); // Bounds check – SAFER
v.front(); // First element
v.back(); // Last element
v.data(); // Raw pointer to underlying array

// Modifiers
v.push_back(x); // Add to end
v.pop_back(); // Remove from end
v.insert(v.begin(), x); // Insert at front:  $O(n)$ 
v.insert(v.begin()+i, x); // Insert at index i
v.insert(v.end(), other.begin(), other.end()); // Append another vector
v.emplace_back(x); // Construct in place (faster)
v.emplace(v.begin()+i, x); // Emplace at position
v.erase(v.begin()+i); // Erase at index
v.erase(v.begin()+i, v.begin()+j); // Erase range [i, j)
v.clear(); // Remove all elements
v.assign(5, 0); // Assign 5 zeros
swap(v1, v2); // Swap two vectors:  $O(1)$ 

// Iterators
v.begin(); v.end(); // Forward iterators
v.rbegin(); v.rend(); // Reverse iterators
v.cbegin(); v.cend(); // Const iterators

// Common patterns
// Erase-remove idiom (remove all occurrences of value)
v.erase(remove(v.begin(), v.end(), target), v.end());

// Remove duplicates from sorted vector
```

```
v.erase(unique(v.begin(), v.end()), v.end());

// Check if element exists
bool exists = find(v.begin(), v.end(), x) != v.end();
// Or for sorted:
bool exists = binary_search(v.begin(), v.end(), x);
```

Interview Tricks

```
// Reverse vector
reverse(v.begin(), v.end());

// Rotate (shift left by k positions)
rotate(v.begin(), v.begin()+k, v.end());

// Fill with value
fill(v.begin(), v.end(), 0);
iota(v.begin(), v.end(), 1); // Fill 1,2,3,4,...

// Min/Max element
auto minIt = min_element(v.begin(), v.end());
auto maxIt = max_element(v.begin(), v.end());
int minVal = *minIt;
int minIdx = minIt - v.begin();

// Sum of elements
int sum = accumulate(v.begin(), v.end(), 0);
long long sum = accumulate(v.begin(), v.end(), 0LL); // Use 0LL!

// Count occurrences
int cnt = count(v.begin(), v.end(), x);
int cnt = count_if(v.begin(), v.end(), [](int x){ return x > 5; });
```

2. array

When to Use

Fixed-size arrays known at compile time. Slightly faster than vector, same as C-arrays but with STL interface.

```
#include <array>
array<int, 5> a = {1, 2, 3, 4, 5};
a.size(); // 5 (compile-time constant)
a.fill(0); // Fill all with 0
sort(a.begin(), a.end());
```

3. deque

Internal Implementation

Double-ended queue backed by a sequence of fixed-size chunks. $O(1)$ insertion/deletion at both ends.

Chunks: [4][5][6] [7][8][9] [10][_][_]
 ←front back→

Operation	Complexity
push<i>front / push</i>back	$O(1)$

Operation	Complexity
pop$\langle i \rangle$front / pop$\langle i \rangle$back	$O(1)$
Access [i]	$O(1)$
Insert/erase middle	$O(n)$

```
#include <deque>
deque<int> dq;
dq.push_front(1); // Add to front
dq.push_back(2);  // Add to back
dq.pop_front();   // Remove from front
dq.pop_back();    // Remove from back
dq.front();       // First element
dq.back();        // Last element
dq[i];            // Random access
```

Interview Use Cases

- Sliding window maximum (monotonic deque)
- BFS level-order traversal (can use queue, but deque is flexible)
- Implement double-ended queue problems

4. list (Doubly Linked List)

When to Use

When you need $O(1)$ insertions/deletions at any known position. Rarely used in FAANG interviews — use vector or deque.

```
#include <list>
list<int> l = {1, 2, 3, 4, 5};
l.push_front(0);
l.push_back(6);
l.pop_front();
l.pop_back();
// No random access: l[i] doesn't work
```

5. map

Internal Implementation

Red-Black Tree (Self-balancing BST). Keys are **always sorted** in ascending order.



Complexity Table

Operation	Complexity
Insert	$O(\log n)$
Delete	$O(\log n)$

Operation	Complexity
Search	$O(\log n)$
Access [key]	$O(\log n)$
Iteration	$O(n)$ in sorted order

Complete API Reference

```
#include <map>
map<string, int> freq;

// Insert/Update
freq["apple"] = 5;           // Insert or update
freq.insert({"banana", 3}); // Insert (won't overwrite if exists)
freq.emplace("cherry", 2);  // Construct in place

// Access
freq["apple"];               // Creates entry if doesn't exist! (DANGER)
freq.at("apple");            // Throws if not found (SAFE)

// Search
auto it = freq.find("apple");
if (it != freq.end()) {
    cout << it->first << " : " << it->second;
}
bool exists = freq.count("apple") > 0; // count returns 0 or 1

// Delete
freq.erase("apple");         // By key
freq.erase(it);              // By iterator

// Size
freq.size();
freq.empty();

// Iteration (sorted by key)
for (auto& [key, val] : freq) {
    cout << key << " : " << val << "\n";
}

// Bounds (useful for range queries)
auto lb = freq.lower_bound("b"); // First key >= "b"
auto ub = freq.upper_bound("c"); // First key > "c"

// Reverse iteration
for (auto it = freq.rbegin(); it != freq.rend(); it++) {
    cout << it->first << " : " << it->second << "\n";
}
```

Interview Patterns

```
// Frequency count
for (int x : nums) freq[x]++;

// Find mode (most frequent)
int maxFreq = max_element(freq.begin(), freq.end(),
    [](auto& a, auto& b){ return a.second < b.second; })->second;

// Group anagrams – sort string as key
map<string, vector<string>> groups;
for (string& s : words) {
    string key = s;
    sort(key.begin(), key.end());
}
```

```
    groups[key].push_back(s);
}
```

6. unordered_map

Internal Implementation

Hash Table with chaining. Average $O(1)$ operations. In worst case (hash collisions), degrades to $O(n)$.

Complexity Table

Operation	Average	Worst
Insert	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$

```
#include <unordered_map>
unordered_map<int, int> umap;

// Same API as map, but:
// - Keys NOT sorted
// -  $O(1)$  average vs  $O(\log n)$ 
// - Use when order doesn't matter

umap[key] = value;
umap.find(key);
umap.count(key);
umap.erase(key);

// Reserve buckets (avoids rehashing, improves performance)
umap.reserve(1000);
umap.max_load_factor(0.25); // Lower = less collisions, more memory
```

Custom Hash (for pairs or custom types)

```
// Hash for pair<int,int>
struct PairHash {
    size_t operator()(const pair<int,int>& p) const {
        return hash<long long>()(((long long)p.first << 32) | p.second);
    }
};
unordered_map<pair<int,int>, int, PairHash> dp;

// Hash for vector<int>
struct VectorHash {
    size_t operator()(const vector<int>& v) const {
        size_t seed = v.size();
        for (auto x : v) {
            seed ^= x + 0x9e3779b9 + (seed << 6) + (seed >> 2);
        }
        return seed;
    }
};
```

When to Use map vs unordered_map

Use <code>map</code> when	Use <code>unordered_map</code> when
Need sorted iteration	Just need fast lookup
Range queries (lower/upper bound)	Frequency counting
Keys are complex objects	Keys are int/string
Predictable performance required	Average case $O(1)$ needed

7. set

Internal Implementation

Red-Black Tree — same as `map` but only stores keys (no values). Always sorted. No duplicates.

Complete API

```
#include <set>
set<int> s = {3, 1, 4, 1, 5, 9}; // {1, 3, 4, 5, 9} – duplicates removed, sorted

s.insert(6);
s.erase(3);
s.find(4); // Returns iterator, or s.end() if not found
s.count(4); // 0 or 1
s.lower_bound(4); // First element >= 4
s.upper_bound(4); // First element > 4
s.size();
s.empty();

// Iteration (sorted)
for (int x : s) cout << x << " ";

// Min and Max
*s.begin(); // Minimum
*s.rbegin(); // Maximum
```

Interview Patterns

```
// Sliding window with ordered set
set<int> window;
// Add/remove elements, get min with *window.begin()

// Next greater/smaller element in sorted order
auto it = s.upper_bound(x); // First > x
if (it != s.end()) cout << *it; // Next greater
if (it != s.begin()) cout << *prev(it); // Previous smaller

// Check if all elements are unique
set<int> unique(v.begin(), v.end());
bool allUnique = unique.size() == v.size();
```

8. unordered_set

When to Use

Fast O(1) membership check, no ordering needed.

```
#include <unordered_set>
unordered_set<int> seen;

seen.insert(x);
seen.count(x);           // 0 or 1
seen.find(x) != seen.end(); // Check membership

// Common interview pattern: seen set for cycle/duplicate detection
for (int x : nums) {
    if (seen.count(x)) return true; // Duplicate found
    seen.insert(x);
}
```

9. multiset

Internal Implementation

Same Red-Black Tree as `set`, but **allows duplicate elements**. Stores all duplicates in sorted order.

```
#include <set> // multiset is in the same header
multiset<int> ms = {3, 1, 4, 1, 5, 9, 2, 6, 5};
// Stored as: {1, 1, 2, 3, 4, 5, 5, 6, 9}

ms.insert(5); // Now has three 5s
ms.erase(ms.find(5)); // Remove ONE 5 (use find + erase)
ms.erase(5); // Remove ALL 5s (dangerous!)
ms.count(5); // Count occurrences

// Min/Max
*ms.begin(); // Minimum
*ms.rbegin(); // Maximum
```

Interview Use Case: Sliding Window Median

```
// Maintain two multisets: lower half (max) and upper half (min)
multiset<int> lower, upper;
// Balance sizes to get median efficiently
```

10. multimap

```
multimap<string, int> mm;
mm.insert({"key", 1});
mm.insert({"key", 2}); // Same key, different values
mm.count("key"); // 2

// Get all values for a key
auto range = mm.equal_range("key");
for (auto it = range.first; it != range.second; it++) {
    cout << it->second << " ";
}
```

11. priority_queue

Internal Implementation

Binary Heap stored as an array. By default a **max-heap** (largest element on top).

Max-Heap:



Complexity Table

Operation	Complexity
push	$O(\log n)$
pop	$O(\log n)$
top	$O(1)$
size/empty	$O(1)$

Complete API

```

#include <queue> // priority_queue is in <queue>

// Max-heap (default)
priority_queue<int> maxPQ;
maxPQ.push(3); maxPQ.push(1); maxPQ.push(4);
maxPQ.top(); // 4 (maximum)
maxPQ.pop(); // Remove maximum

// Min-heap
priority_queue<int, vector<int>, greater<int>> minPQ;
minPQ.push(3); minPQ.push(1); minPQ.push(4);
minPQ.top(); // 1 (minimum)

// Custom comparator with lambda
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.second > b.second; // Min-heap by second element
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);

// Custom comparator with struct
struct Compare {
    bool operator()(pair<int,int> a, pair<int,int> b) {
        return a.second > b.second; // Min-heap by second
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Compare> pq;

// Build from vector
vector<int> v = {3,1,4,1,5,9};
priority_queue<int> pq(v.begin(), v.end());

```

Interview Patterns

```

// K largest elements
priority_queue<int, vector<int>, greater<int>> minPQ;
for (int x : nums) {
    minPQ.push(x);
    if (minPQ.size() > k) minPQ.pop(); // Keep only k elements
}

```

```
// minPQ now contains k largest, top() is kth largest

// Dijkstra's algorithm
priority_queue<pair<int,int>;, vector<pair<int,int>>&, greater<pair<int,int>>
pq.push({0, src}); // {dist, node}
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    for (auto [v, w] : graph[u]) {
        if (d + w < dist[v]) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}

// Merge K sorted arrays
// Use min-heap with {value, array_index, element_index}
```

12. stack

Internal Implementation

Adapter over `deque` (or `vector`). **LIFO** — Last In, First Out.

```
#include <stack>
stack<int> st;

st.push(1); // Push onto top
st.push(2);
st.top(); // Access top: O(1) — returns 2
st.pop(); // Remove top: O(1)
st.empty(); // Check if empty
st.size(); // Number of elements

// Interview patterns using stack
// 1. Balanced parentheses
// 2. Next greater element
// 3. Monotonic stack problems
// 4. Expression evaluation
// 5. Implement queue using stacks
```

13. queue

Internal Implementation

Adapter over `deque`. **FIFO** — First In, First Out.

```
#include <queue>
queue<int> q;

q.push(1); // Enqueue at back
q.push(2);
q.front(); // Access front: O(1) — returns 1
q.back(); // Access back: O(1)
q.pop(); // Dequeue from front: O(1)
q.empty();
q.size();
// BFS template using queue
```

```

queue<int> bfsQ;
bfsQ.push(start);
while (!bfsQ.empty()) {
    int node = bfsQ.front();
    bfsQ.pop();
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            bfsQ.push(neighbor);
        }
    }
}

```

14. pair & tuple

pair

```

#include <utility>
pair<int, string> p = {42, "hello"};
p.first;           // 42
p.second;          // "hello"

// Make pair
auto p2 = make_pair(1, 2);

// Pair comparison (lexicographic by default)
pair<int,int> a = {1, 2}, b = {1, 3};
a < b; // true (compares first, then second)

// Common usage: coordinate, graph edge, sorting by multiple keys
vector<pair<int,int>> edges; // {cost, node}
sort(edges.begin(), edges.end()); // Sorts by cost, then node

// Structured bindings (C++17)
auto [x, y] = p; // x=42, y="hello"
for (auto& [key, val] : myMap) { /* ... */ }

```

tuple

```

#include <tuple>
tuple<int, string, double> t = {1, "hello", 3.14};

get<0>(t); // 1
get<1>(t); // "hello"
get<2>(t); // 3.14

auto t2 = make_tuple(1, "world", 2.71);

// Structured binding
auto [a, b, c] = t;

// Tie (assign tuple elements to variables)
int x; string s; double d;
tie(x, s, d) = t;

// Useful for returning 3+ values
tuple<int,int,int> solve() {
    return {min_val, max_val, count};
}
auto [mn, mx, cnt] = solve();

```

15. Algorithms Library

```

#include <algorithm>
#include <numeric>

// ===== Sorting =====
sort(v.begin(), v.end()); // O(n log n)
sort(v.begin(), v.end(), greater<int>()); // Descending
stable_sort(v.begin(), v.end()); // Stable sort
partial_sort(v.begin(), v.begin()+k, v.end()); // Only sort first k
nth_element(v.begin(), v.begin()+k, v.end()); // kth element in O(n) avg

// ===== Searching =====
binary_search(v.begin(), v.end(), x); // O(log n), requires sorted
lower_bound(v.begin(), v.end(), x); // First >= x
upper_bound(v.begin(), v.end(), x); // First > x
equal_range(v.begin(), v.end(), x); // Range of equal elements
find(v.begin(), v.end(), x); // Linear search
find_if(v.begin(), v.end(), pred); // Find by predicate

// ===== Min/Max =====
min(a, b);
max(a, b);
min({a, b, c, d}); // Multiple values
max_element(v.begin(), v.end());
min_element(v.begin(), v.end());
minmax_element(v.begin(), v.end()); // Returns pair of iterators
clamp(x, lo, hi); // Clamp to [lo, hi]

// ===== Permutations =====
next_permutation(v.begin(), v.end()); // Next lexicographic permutation
prev_permutation(v.begin(), v.end()); // Previous permutation

// Generate all permutations:
sort(v.begin(), v.end());
do {
    // process permutation
} while (next_permutation(v.begin(), v.end()));

// ===== Transform & Fill =====
fill(v.begin(), v.end(), 0); // Fill with value
iota(v.begin(), v.end(), 1); // 1, 2, 3, 4, ...
transform(v.begin(), v.end(), v.begin(), [](int x){ return x*2; }); // Double each

// ===== Set Operations (requires sorted input) =====
set_intersection(a.begin(), a.end(), b.begin(), b.end(), out.begin());
set_union(a.begin(), a.end(), b.begin(), b.end(), out.begin());
set_difference(a.begin(), a.end(), b.begin(), b.end(), out.begin());

// ===== Numeric =====
accumulate(v.begin(), v.end(), 0); // Sum
accumulate(v.begin(), v.end(), 1, multiplies<int>()); // Product
partial_sum(v.begin(), v.end(), prefix.begin()); // Prefix sums
adjacent_difference(v.begin(), v.end(), diff.begin()); // Differences
inner_product(a.begin(), a.end(), b.begin(), 0); // Dot product
gcd(a, b); // GCD (C++17)
lcm(a, b); // LCM (C++17)

// ===== Manipulation =====
reverse(v.begin(), v.end());
rotate(v.begin(), v.begin()+k, v.end()); // Left rotate by k
random_shuffle(v.begin(), v.end()); // Shuffle
shuffle(v.begin(), v.end(), mt19937(random_device{}())); // C++11 shuffle
unique(v.begin(), v.end()); // Remove consecutive dups (sorted)

```

```

remove(v.begin(), v.end(), x); // Move to end, returns new e
nd
replace(v.begin(), v.end(), old_val, new_val);

// ===== Count =====
count(v.begin(), v.end(), x); // Count occurrences
count_if(v.begin(), v.end(), pred); // Count by predicate

// ===== Check =====
all_of(v.begin(), v.end(), pred); // All satisfy predicate
any_of(v.begin(), v.end(), pred); // Any satisfies predicate
none_of(v.begin(), v.end(), pred); // None satisfies predicate
is_sorted(v.begin(), v.end()); // Check if sorted

```

16. STL Optimization Tricks

1. Reserve Capacity Before Bulk Insertions

```

vector<int> v;
v.reserve(n); // Avoids O(n log n) reallocations total
for (int i = 0; i < n; i++) v.push_back(i);

```

2. `emplace_back` vs `push_back`

```

// push_back: constructs then copies/moves
v.push_back(pair<int,int>(1, 2));

// emplace_back: constructs in place (no copy)
v.emplace_back(1, 2); // Faster for complex objects

```

3. Avoid `[]` Operator in Maps When Checking

```

// BAD: creates entry if not found!
if (myMap["key"] == 0) { ... }

// GOOD:
if (myMap.count("key") && myMap["key"] == 0) { ... }
// OR:
if (auto it = myMap.find("key"); it != myMap.end()) { ... }

```

4. Use `unordered_map::reserve`

```

unordered_map<int,int> freq;
freq.reserve(n); // Avoids rehashing
freq.max_load_factor(0.25); // More buckets = fewer collisions

```

5. String Operations

```

// SLOW: string concatenation in loop
string result = "";
for (string& s : v) result += s; // O(n²) total!

// FAST: use stringstream or join then build
ostringstream oss;
for (string& s : v) oss << s;
string result = oss.str();

```

6. Sorting Custom Objects

```

// Comparator function

```

```
bool cmp(const pair<int,int>& a, const pair<int,int>& b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second; // Tie-break: second descending
}
sort(v.begin(), v.end(), cmp);

// Lambda (cleaner for interviews)
sort(v.begin(), v.end(), [](const pair<int,int>& a, const pair<int,int>& b) {
    return a.first != b.first ? a.first < b.first : a.second > b.second;
});
```

7. Two-Pass Iteration Pattern

```
// Count elements, then process in one pass
unordered_map<int,int> freq;
for (int x : nums) freq[x]++;
for (int x : nums) {
    if (freq[x] == 1) return x; // First unique
}
```

STL Cheat Sheet — Quick Reference

```
// ===== CONTAINERS =====
// Sequential
vector<T>; // Dynamic array, O(1) random access
deque<T>; // Double-ended, O(1) both ends
list<T>; // Doubly linked, O(1) insert/delete at pointer
array<T,N>; // Fixed size, stack allocated

// Associative (sorted, tree-based)
map<K,V>; // K-V pairs, sorted by K
set<K>; // Unique keys, sorted
multimap<K,V>; // Sorted, duplicate keys allowed
multiset<K>; // Sorted, duplicates allowed

// Unordered (hash-based)
unordered_map<K,V>; // Hash table K-V
unordered_set<K>; // Hash table keys

// Adaptors
stack<T>; // LIFO
queue<T>; // FIFO
priority_queue<T>; // Max-heap by default

// ===== KEY OPERATIONS =====
// All sorted containers: lower_bound(), upper_bound()
// All containers: size(), empty(), clear()
// All sequence containers: begin(), end(), front(), back()
// Insert: push_back(), insert(), emplace_back(), emplace()
// Remove: pop_back(), erase(), pop(), pop_front()
```

<i>Next: <i><u>Section 3 — DSA Foundations</u></i></i>

Section 3: DSA Foundations

Complexity Analysis, Recursion, Sorting & Searching

****Goal:**** Build an unshakeable foundation in algorithm analysis, recursion thinking, and the core algorithms that appear in every interview.

Table of Contents

1. [Complexity Analysis](#)
2. [Recursion](#)
3. [Sorting Algorithms](#)
4. [Searching Algorithms](#)
5. [Divide and Conquer](#)
6. [Recursion Trees](#)

1. Complexity Analysis

Big-O Notation

Big-O describes the **upper bound** of an algorithm's growth rate as input size $n \rightarrow \infty$. It ignores constants and lower-order terms.

$O(1)$ $\ll$ $O(\log n)$ $\ll$ $O(\sqrt{n})$ $\ll$ $O(n)$ $\ll$ $O(n \log n)$
 $\ll$ $O(n^2)$ $\ll$ $O(n^3)$ $\ll$ $O(2^n)$ $\ll$ $O(n!)$

Common Complexities with Real Examples

Complexity	Name	Example Algorithm	n=10	n=100	n=1000
$O(1)$	Constant	Array access, HashMap lookup	1	1	1
$O(\log n)$	Logarithmic	Binary search	3	7	10
$O(\sqrt{n})$	Square root	Sieve of Eratosthenes	3	10	32
$O(n)$	Linear	Linear scan, Single loop	10	100	1000
$O(n \log n)$	Linearithmic	Merge sort, Heap sort	33	664	9966
$O(n^2)$	Quadratic	Bubble/Selection sort, Nested loops	100	10000	1M
$O(n^3)$	Cubic	Floyd-Warshall	1000	1M	1B
$O(2^n)$	Exponential	Subset generation, naive recursion	1024	2^{100}	∞

Complexity	Name	Example Algorithm	n=10	n=100	n=1000
$O(n!)$	Factorial	Permutation generation	3628800	∞	∞

FAANG Acceptable Complexity by Input Size

$n \leq 10$: $O(n!)$ or $O(2^n)$ – Backtracking, brute force OK
 $n \leq 20$: $O(2^n)$ – Bitmask DP
 $n \leq 100$: $O(n^3)$ – Triple nested loops
 $n \leq 1,000$: $O(n^2)$ – Double nested loops
 $n \leq 10,000$: $O(n^2 \log n)$ – Careful nested loops
 $n \leq 100,000$: $O(n \log n)$ – Sort + BST operations
 $n \leq 1,000,000$: $O(n)$ or $O(n \log n)$ – Linear algorithms
 $n \leq 10,000,000$: $O(n)$ or $O(n \log n)$ – Optimized linear
 $n \geq 10,000,000$: $O(\log n)$ or $O(1)$ – Binary search, math

How to Calculate Complexity

```
// Rule 1: Drop constants
O(3n) = O(n)
O(n/2) = O(n)

// Rule 2: Drop lower-order terms
O(n^2 + n) = O(n^2)
O(n log n + n) = O(n log n)

// Rule 3: Sequential blocks – ADD
for (int i = 0; i < n; i++) { } // O(n)
for (int i = 0; i < n; i++) { } // O(n)
// Total: O(n) + O(n) = O(2n) = O(n)

// Rule 4: Nested blocks – MULTIPLY
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { // O(n) * O(n) = O(n^2)
    }
}

// Rule 5: Halving the input – LOG
int i = n;
while (i > 1) {
    i /= 2; // O(log n)
}

// Rule 6: Different inputs – different variables
for (int i = 0; i < n; i++) { } // O(n)
for (int i = 0; i < m; i++) { } // O(m)
// Total: O(n + m), NOT O(n^2)!

// Rule 7: Recursive – use recurrence relation
// T(n) = 2T(n/2) + O(n) → O(n log n) (Merge sort)
// T(n) = T(n-1) + O(1) → O(n) (Linear recursion)
// T(n) = 2T(n-1) + O(1) → O(2^n) (Exponential)
```

Space Complexity

```
// O(1) – constant extra space
int maxSum = 0;
for (int x : nums) maxSum = max(maxSum, x);

// O(n) – linear extra space
vector<int> result(n); // Storing n elements
unordered_map<int,int> freq; // Up to n entries
```

```
// O(n) – recursion stack space
// Recursive calls use stack space = depth of recursion
void dfs(int n) { // O(n) stack space for n recursive calls
    if (n == 0) return;
    dfs(n-1);
}

// O(log n) – balanced tree recursion
void mergeSort(vector<int>& v, int l, int r) { // O(log n) stack depth
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(v, l, mid);
    mergeSort(v, mid+1, r);
    merge(v, l, mid, r);
}
```

Master Theorem (Quick Reference)

For $T(n) = aT(n/b) + O(n^d)$:

- If $d > \log_b(a)$: $T(n) = O(n^d)$
- If $d = \log_b(a)$: $T(n) = O(n^d \cdot \log n)$
- If $d < \log_b(a)$: $T(n) = O(n^{\log_b(a)})$

Merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow a=2, b=2, d=1 \rightarrow d=\log_2 2=1 \rightarrow O(n \log n)$
 Binary search: $T(n) = T(n/2) + O(1) \rightarrow a=1, b=2, d=0 \rightarrow d=\log_2 1=0 \rightarrow O(\log n)$

2. Recursion

The Recursion Mental Model

Every recursive function has:

1. **Base case** — when to stop
2. **Recursive case** — break into smaller subproblem
3. **Trust the recursion** — assume recursive call works correctly

```
// Example: Factorial
int factorial(int n) {
    if (n <= 1) return 1; // Base case
    return n * factorial(n - 1); // Recursive case
}

// Call stack visualization for factorial(4):
// factorial(4) → 4 * factorial(3)
//                → 3 * factorial(2)
//                → 2 * factorial(1)
//                → 1 (base case)
//                ← 2 * 1 = 2
//                ← 3 * 2 = 6
//                ← 4 * 6 = 24
```

Fibonacci & Memoization

```
// Naive: O(2^n)
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// Memoized: O(n) time, O(n) space
```

```

unordered_map<int,int> memo;
int fib(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}

// Bottom-up DP: O(n) time, O(1) space
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b; b = c;
    }
    return b;
}

```

Recursive Patterns in Interviews

```

// 1. Subsets / Power Set
void subsets(vector<int>& nums, int idx, vector<int>& current, vector<vector<int>>& result) {
    result.push_back(current);
    for (int i = idx; i < nums.size(); i++) {
        current.push_back(nums[i]);
        subsets(nums, i+1, current, result);
        current.pop_back(); // Backtrack
    }
}

// 2. Permutations
void permutations(vector<int>& nums, vector<bool>& used, vector<int>& current, vector<vector<int>>& result) {
    if (current.size() == nums.size()) {
        result.push_back(current);
        return;
    }
    for (int i = 0; i < nums.size(); i++) {
        if (!used[i]) {
            used[i] = true;
            current.push_back(nums[i]);
            permutations(nums, used, current, result);
            current.pop_back();
            used[i] = false;
        }
    }
}

// 3. Tree traversal
void inorder(TreeNode* node, vector<int>& result) {
    if (!node) return;
    inorder(node->left, result);
    result.push_back(node->val);
    inorder(node->right, result);
}

```

3. Sorting Algorithms

Sorting Overview

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes
STL <code>std::sort</code>	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No

Key Insight: `std::sort` uses **introsort** (hybrid of quicksort + heapsort + insertion sort) — never degrades to $O(n^2)$.

Merge Sort — Implementation

```
// O(n log n) time, O(n) space – STABLE
void merge(vector<int>& arr, int l, int mid, int r) {
    vector<int> left(arr.begin()+l, arr.begin()+mid+1);
    vector<int> right(arr.begin()+mid+1, arr.begin()+r+1);

    int i = 0, j = 0, k = l;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else arr[k++] = right[j++];
    }
    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2; // Avoids overflow: NOT (l+r)/2
    mergeSort(arr, l, mid);
    mergeSort(arr, mid+1, r);
    merge(arr, l, mid, r);
}

// Interview use case: Count inversions
long long merge_count(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long count = 0;
    count += merge_count(arr, l, mid);
    count += merge_count(arr, mid+1, r);
    // During merge, count pairs
    vector<int> left(arr.begin()+l, arr.begin()+mid+1);
    vector<int> right(arr.begin()+mid+1, arr.begin()+r+1);
    int i = 0, j = 0, k = l;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else {
            count += left.size() - i; // All remaining in left > right[j]
            arr[k++] = right[j++];
        }
    }
}
```



```

    }
    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
    return count;
}

```

Quick Sort — Implementation

```

// O(n log n) average, O(n²) worst case – NOT stable
int partition(vector<int>& arr, int l, int r) {
    int pivot = arr[r]; // Pivot = last element
    int i = l - 1;
    for (int j = l; j < r; j++) {
        if (arr[j] <= pivot) {
            swap(arr[++i], arr[j]);
        }
    }
    swap(arr[i+1], arr[r]);
    return i + 1;
}

void quickSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int pi = partition(arr, l, r);
    quickSort(arr, l, pi-1);
    quickSort(arr, pi+1, r);
}

// Randomized quicksort (avoids worst case on sorted input)
int randomPartition(vector<int>& arr, int l, int r) {
    int randIdx = l + rand() % (r - l + 1);
    swap(arr[randIdx], arr[r]);
    return partition(arr, l, r);
}

```

Counting Sort — Linear Sorting

```

// O(n + k) time, O(k) space – requires bounded integer range
void countSort(vector<int>& arr) {
    int maxVal = *max_element(arr.begin(), arr.end());
    vector<int> count(maxVal + 1, 0);

    for (int x : arr) count[x]++;

    int idx = 0;
    for (int i = 0; i <= maxVal; i++) {
        while (count[i]-- > 0) arr[idx++] = i;
    }
}

```

4. Searching Algorithms

Linear Search

```

// O(n) – works on unsorted arrays
int linearSearch(vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

```

Binary Search — The Most Important Search

```
// O(log n) — REQUIRES sorted array
// Template 1: Classic
int binarySearch(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Prevents overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// Template 2: Find leftmost occurrence (lower_bound)
int lowerBound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] < target) left = mid + 1;
        else right = mid;
    }
    return left; // First index where arr[i] >= target
}

// Template 3: Find rightmost occurrence (upper_bound)
int upperBound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] <= target) left = mid + 1;
        else right = mid;
    }
    return left; // First index where arr[i] > target
}

// Template 4: Search on answer (binary search on result)
// "Find minimum k such that condition(k) is true"
int binarySearchOnAnswer(int lo, int hi) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (condition(mid)) hi = mid; // Shrink right
        else lo = mid + 1; // Shrink left
    }
    return lo; // Minimum satisfying k
}

// Search in rotated sorted array
int searchRotated(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;

        if (arr[left] <= arr[mid]) { // Left half is sorted
            if (arr[left] <= target && target < arr[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // Right half is sorted
            if (arr[mid] < target && target <= arr[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

Binary Search Application Patterns

```
// 1. Find peak element
int findPeak(vector<int>& arr) {
    int l = 0, r = arr.size() - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (arr[mid] < arr[mid+1]) l = mid + 1;
        else r = mid;
    }
    return l;
}

// 2. Square root (integer)
int mySqrt(int x) {
    long long l = 0, r = x;
    while (l < r) {
        long long mid = l + (r - l + 1) / 2; // Upper mid
        if (mid * mid <= x) l = mid;
        else r = mid - 1;
    }
    return l;
}

// 3. Koko eating bananas (binary search on answer)
bool canFinish(vector<int>& piles, int k, int h) {
    long long hours = 0;
    for (int p : piles) hours += (p + k - 1) / k;
    return hours <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canFinish(piles, mid, h)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

5. Divide and Conquer

Template

```
DivideAndConquer(problem):
    if problem is small enough:
        solve directly (base case)
    else:
        divide into subproblems
        recursively solve each subproblem
        combine results
```

Classic Problems

```
// Maximum Subarray Sum (Kadane's uses DP, but D&C version:)
int maxCrossing(vector<int>& arr, int l, int mid, int r) {
    int leftSum = INT_MIN, rightSum = INT_MIN;
    int sum = 0;
    for (int i = mid; i >= l; i--) {
        sum += arr[i];
        leftSum = max(leftSum, sum);
    }
    sum = 0;
    for (int i = mid+1; i <= r; i++) {
        sum += arr[i];
        rightSum = max(rightSum, sum);
    }
    return max(leftSum, rightSum);
}
```

```

        for (int i = mid+1; i <= r; i++) {
            sum += arr[i];
            rightSum = max(rightSum, sum);
        }
        return leftSum + rightSum;
    }

int maxSubarrayDC(vector<int>& arr, int l, int r) {
    if (l == r) return arr[l];
    int mid = l + (r - l) / 2;
    return max({maxSubarrayDC(arr, l, mid),
                maxSubarrayDC(arr, mid+1, r),
                maxCrossing(arr, l, mid, r)});
}

// Closest Pair of Points – O(n log n)
// Binary Exponentiation – O(log n)
long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

```

6. Recursion Trees

How to Draw a Recursion Tree

For $T(n) = 2T(n/2) + O(n)$ (Merge Sort):

```

Level 0:          n                      ← Work: n
Level 1:      n/2      n/2              ← Work: n/2 + n/2 = n
Level 2:    n/4  n/4  n/4  n/4          ← Work: n
...
Level log n: 1  1  1  1  1  1  1  1    ← Work: n

```

```

Total levels: log n
Work per level: n
Total: O(n log n)

```

For $T(n) = 2T(n-1) + O(1)$ (Naive Fibonacci):

```

Level 0:          fib(n)                  ← 1 call
Level 1:      fib(n-1)  fib(n-2)          ← 2 calls
Level 2:    fib(n-2) fib(n-3) ...          ← 4 calls
...
Level n:    1 1 1 1 1 1 ... (2^n calls) ← 2^n calls

```

Total: $O(2^n)$

Common Recurrences

Recurrence	Example	Complexity
$T(n) = T(n/2) + O(1)$	Binary search	$O(\log n)$
$T(n) = T(n-1) + O(1)$	Linear recursion	$O(n)$
$T(n) = 2T(n/2) + O(n)$	Merge sort	$O(n \log n)$

Recurrence	Example	Complexity
$T(n) = 2T(n/2) + O(1)$	Tree traversal	$O(n)$
$T(n) = T(n-1) + O(n)$	Selection sort (recursive)	$O(n^2)$
$T(n) = 2T(n-1) + O(1)$	Naive Fibonacci	$O(2^n)$
$T(n) = T(\sqrt{n}) + O(1)$	Integer factorization	$O(\log \log n)$

Interview: Complexity Analysis Questions

Q: What is the complexity of this code?

```
// Problem 1:
for (int i = 1; i <= n; i *= 2) { // i = 1, 2, 4, 8, ... → O(log n) iterations
    for (int j = 0; j < n; j++) { // O(n) each
        // O(1) work
    }
}
// Answer: O(n log n)

// Problem 2: (tricky!)
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) { // j starts from i, NOT 0
        // O(1) work
    }
}
// Total iterations: n + (n-1) + (n-2) + ... + 1 = n(n+1)/2 = O(n^2)

// Problem 3:
void func(int n) {
    if (n <= 1) return;
    func(n/2);
    func(n/2); // Two recursive calls, each on n/2
    for (int i = 0; i < n; i++) { } // O(n) work
}
// T(n) = 2T(n/2) + O(n) → O(n log n) by Master Theorem

// Problem 4: (tricky!)
int i = n;
while (i > 0) i = i / 2; // O(log n)
// ... then
for (int j = 0; j < i; j++) { } // j never executes! i=0 after loop
// Total: O(log n)
```

Key Formulas for Interviews

Geometric series: $1 + 2 + 4 + \dots + 2^k = 2^{(k+1)} - 1 \approx 2 * 2^k$
 Arithmetic series: $1 + 2 + \dots + n = n(n+1)/2 \approx n^2/2$
 Log identity: $\log(ab) = \log(a) + \log(b)$
 Log identity: $\log(a^b) = b * \log(a)$
 $\log_2(n) \approx 10$ when $n = 1024$ ($\approx 10^3$)
 $\log_2(10^6) \approx 20$
 $\log_2(10^9) \approx 30$

Next: [Section 4 — Complete DSA Pattern System](#)

Section 4: Complete DSA Pattern System

15 Core Patterns for FAANG Interviews

*****The Meta-Skill:**** Recognizing which pattern to apply is more important than memorizing solutions. This section gives you a ****pattern recognition radar**** for every interview problem.*

Pattern Recognition Cheatsheet

Signal in Problem	Pattern
"subarray/substring of fixed length k"	Sliding Window (Fixed)
"longest/shortest subarray satisfying condition"	Sliding Window (Variable)
"sorted array, find pair summing to X"	Two Pointers
"search in sorted/rotated array"	Binary Search
"subarray sum equals k"	Prefix Sum
"count/check duplicates, frequency"	HashMap/HashSet
"matching parentheses, next greater/smaller"	Stack
"binary tree path, traversal"	Tree DFS/BFS
"shortest path, connected components, grid"	Graph BFS/DFS
"overlapping subproblems, optimization"	Dynamic Programming
"generate all combinations/permutations"	Backtracking
"k largest/smallest, median of stream"	Heap
"merge intervals, meeting rooms"	Intervals
"minimum coins, activity selection"	Greedy
"prefix search, word dictionary"	Trie
"XOR, check power of 2, bit count"	Bit Manipulation

Pattern 1: Sliding Window

Concept

Move a window across an array/string. Avoid recomputing from scratch — add new element, remove old element.

```
arr = [1, 3, -1, -3, 5, 3, 6, 7]
  1  3  -1  -3  5  3  6  7   → window [0,2], max=3
    1  3  -1  -3  5  3  6  7   → window [1,3], max=3
      1  3  -1  -3  5  3  6  7   → window [2,4], max=5
```

Templates

```
// Template 1: Fixed-size window
int maxSumWindow(vector<int>& arr, int k) {
    int windowSum = 0;
```

```

    for (int i = 0; i < k; i++) windowSum += arr[i];

    int maxSum = windowSum;
    for (int i = k; i < arr.size(); i++) {
        windowSum += arr[i] - arr[i-k]; // Slide: add new, remove old
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}

// Template 2: Variable-size window (expand/shrink)
int longestSubstringKDistinct(string& s, int k) {
    unordered_map<char,int> freq;
    int left = 0, result = 0;

    for (int right = 0; right < s.size(); right++) {
        freq[s[right]]++; // Expand window

        while (freq.size() > k) { // Window condition violated
            freq[s[left]]--; // Shrink from left
            if (freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }
        result = max(result, right - left + 1);
    }
    return result;
}

// Template 3: Sliding window maximum (Monotonic Deque)
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // Stores INDICES, decreasing order of values
    vector<int> result;

    for (int i = 0; i < nums.size(); i++) {
        // Remove elements outside window
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // Remove smaller elements (maintain decreasing order)
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();

        dq.push_back(i);
        if (i >= k-1) result.push_back(nums[dq.front()]); // Max = front
    }
    return result;
}

```

Common Problems

Problem	Pattern	Key Insight
Max sum subarray of size k	Fixed window	Sum = add right, remove left
Longest substring without repeating	Variable window	Use set/map to track
Minimum window substring	Variable window	Expand until valid, shrink
Sliding window maximum	Monotonic deque	Deque stores indices in decreasing order
Longest subarray with sum $\leq k$	Variable window	Shrink when sum exceeds k

Dry Run: Minimum Window Substring

s = "ADOBECODEBANC", t = "ABC"
 need: {A:1, B:1, C:1}, formed=0, required=3

```

R=0: A, freq{A:1}, formed=1
R=1: D, freq{D:1}, formed=1
R=2: O, ...
R=3: B, freq{B:1}, formed=2
R=4: E, ...
R=5: C, freq{C:1}, formed=3 ← All formed!
    → window="ADOBEC", len=6, minLen=6
    Shrink: L=0 A→freq{A:0}, formed=2, L=1
R=6: O, formed=2
...
Continue until optimal window "BANC" found

string minWindow(string s, string t) {
    unordered_map<char,int> need, have;
    for (char c : t) need[c]++;

    int formed = 0, required = need.size();
    int left = 0, minLen = INT_MAX, minStart = 0;

    for (int right = 0; right < s.size(); right++) {
        have[s[right]]++;
        if (need.count(s[right]) && have[s[right]] == need[s[right]])
            formed++;

        while (formed == required) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }
            have[s[left]]--;
            if (need.count(s[left]) && have[s[left]] < need[s[left]])
                formed--;
            left++;
        }
    }
    return minLen == INT_MAX ? "" : s.substr(minStart, minLen);
}

```

Pattern 2: Two Pointers

Concept

Use two pointers that move toward each other or in the same direction to avoid $O(n^2)$ nested loops.

Templates

```

// Template 1: Opposite ends (sorted array)
int twoSum(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return {left, right};
        else if (sum < target) left++;
        else right--;
    }
    return {};
}

// Template 2: Same direction (fast/slow pointers)
// Remove duplicates from sorted array
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;
    int slow = 0;

```



```

        for (int fast = 1; fast < nums.size(); fast++) {
            if (nums[fast] != nums[slow]) {
                nums[++slow] = nums[fast];
            }
        }
        return slow + 1;
    }

// Template 3: Floyd's cycle detection
bool hasCycle(ListNode* head) {
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

// 3Sum problem
vector<vector<int>>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>>> result;

    for (int i = 0; i < nums.size()-2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue; // Skip duplicates

        int left = i+1, right = nums.size()-1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left+1]) left++;
                while (left < right && nums[right] == nums[right-1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}

```

Pattern 3: Binary Search

Concept

Eliminate half the search space each iteration. Works on any **monotonically changing** condition.

Key Question: "What property changes from false $\rightarrow$ true (or true $\rightarrow$ false) as we move across the search space?"

Templates

```

// Classic binary search
int binarySearch(vector<int>& arr, int target) {
    int l = 0, r = arr.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
}

```

```

    }
    return -1;
}

// Binary search on answer – find minimum satisfying condition
// "Is it possible to achieve X?" must be monotonic (F F F T T T)
int binarySearchOnAnswer() {
    int lo = min_possible, hi = max_possible;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (isPossible(mid)) hi = mid; // mid might be answer, exclude nothing
        else lo = mid + 1; // mid doesn't work, exclude it
    }
    return lo;
}

```

Problem: Split Array Largest Sum (Hard)

```

// Binary search on answer: what is the minimum possible largest sum?
bool canSplit(vector<int>& nums, int maxSum, int m) {
    int parts = 1, currSum = 0;
    for (int x : nums) {
        if (currSum + x > maxSum) { parts++; currSum = 0; }
        currSum += x;
    }
    return parts <= m;
}

int splitArray(vector<int>& nums, int m) {
    int lo = *max_element(nums.begin(), nums.end());
    int hi = accumulate(nums.begin(), nums.end(), 0);

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canSplit(nums, mid, m)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

```

Pattern 4: Prefix Sum

Concept

Precompute cumulative sums to answer range sum queries in $O(1)$.

```

arr    = [1, 2, 3, 4, 5]
prefix = [0, 1, 3, 6, 10, 15]

```

```

sum(l, r) = prefix[r+1] - prefix[l]
sum(1, 3) = prefix[4] - prefix[1] = 10 - 1 = 9 ✓ (2+3+4=9)

```

Templates

```

// 1D Prefix Sum
vector<int> buildPrefix(vector<int>& arr) {
    int n = arr.size();
    vector<int> prefix(n+1, 0);
    for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + arr[i];
    return prefix;
}

int rangeSum(vector<int>& prefix, int l, int r) {

```

```

    return prefix[r+1] - prefix[l]; // sum of arr[l..r]
}

// 2D Prefix Sum
vector<vector<int>>> build2DPrefix(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    vector<vector<int>>> p(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            p[i][j] = grid[i-1][j-1] + p[i-1][j] + p[i][j-1] - p[i-1][j-1];
    return p;
}

int query2D(vector<vector<int>>& p, int r1, int c1, int r2, int c2) {
    return p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1];
}

// Subarray sum equals k (using prefix sum + hashmap)
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> prefixCount;
    prefixCount[0] = 1;
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x;
        count += prefixCount[sum - k]; // How many prefixes end at sum-k?
        prefixCount[sum]++;
    }
    return count;
}

```

Pattern 5: HashMap / HashSet

Concept

Trade space for time. Convert $O(n^2)$ lookups into $O(n)$ with $O(1)$ hash operations.

Templates

```

// Two Sum – classic hashmap
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> seen; // value → index
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement)) return {seen[complement], i};
        seen[nums[i]] = i;
    }
    return {};
}

// Frequency count pattern
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for (int x : nums) freq[x]++;

    // Bucket sort: index = frequency
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto& [val, cnt] : freq) buckets[cnt].push_back(val);

    vector<int> result;
    for (int i = buckets.size()-1; i >= 0 && result.size() < k; i--)
        for (int x : buckets[i]) result.push_back(x);
    return result;
}

```

```
// Group anagrams
vector<vector<string>>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>>> groups;
    for (string& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        groups[key].push_back(s);
    }
    vector<vector<string>>> result;
    for (auto& [key, v] : groups) result.push_back(v);
    return result;
}

// Longest consecutive sequence – O(n) with hashset
int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(), nums.end());
    int maxLen = 0;
    for (int n : s) {
        if (!s.count(n-1)) { // Only start from sequence beginning
            int curr = n, len = 1;
            while (s.count(curr+1)) { curr++; len++; }
            maxLen = max(maxLen, len);
        }
    }
    return maxLen;
}
```

Pattern 6: Stack Patterns

Concept

Stack is perfect for problems requiring **last-seen** information or **monotonic** ordering.

Monotonic Stack Template

```
// Next Greater Element
vector<int> nextGreater(vector<int>& arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> st; // Stack of indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] < arr[i]) {
            result[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }
    return result;
}

// Largest Rectangle in Histogram
int largestRectangle(vector<int>& heights) {
    stack<int> st;
    int maxArea = 0;
    heights.push_back(0); // Sentinel to flush stack

    for (int i = 0; i < heights.size(); i++) {
        while (!st.empty() && heights[st.top()] > heights[i]) {
            int h = heights[st.top()]; st.pop();
            int w = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        st.push(i);
    }
}
```

```

        }
        st.push(i);
    }
    return maxArea;
}

// Valid Parentheses
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') st.push(c);
        else {
            if (st.empty()) return false;
            char top = st.top(); st.pop();
            if ((c == ')' && top != '(') ||
                (c == '}' && top != '{') ||
                (c == ']' && top != '[')) return false;
        }
    }
    return st.empty();
}

// Daily Temperatures – next warmer day
vector<int> dailyTemperatures(vector<int>& T) {
    int n = T.size();
    vector<int> result(n, 0);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && T[st.top()] < T[i]) {
            result[st.top()] = i - st.top();
            st.pop();
        }
        st.push(i);
    }
    return result;
}

```

Pattern 7: Tree Patterns

Tree Node Definition

```

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

```

DFS Traversals

```

// Inorder (Left → Root → Right) – gives sorted order for BST
void inorder(TreeNode* root, vector<int>& result) {
    if (!root) return;
    inorder(root->left, result);
    result.push_back(root->val);
    inorder(root->right, result);
}

// Preorder (Root → Left → Right) – used to serialize tree
void preorder(TreeNode* root, vector<int>& result) {
    if (!root) return;
    result.push_back(root->val);
    preorder(root->left, result);
}

```

```

        preorder(root->right, result);
    }

// Postorder (Left → Right → Root) – used for deletion, expression trees
void postorder(TreeNode* root, vector<int>& result) {
    if (!root) return;
    postorder(root->left, result);
    postorder(root->right, result);
    result.push_back(root->val);
}

// Iterative inorder (common follow-up question)
vector<int> inorderIterative(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr || !st.empty()) {
        while (curr) { st.push(curr); curr = curr->left; }
        curr = st.top(); st.pop();
        result.push_back(curr->val);
        curr = curr->right;
    }
    return result;
}

```

BFS (Level Order)

```

vector<vector<int>>> levelOrder(TreeNode* root) {
    if (!root) return {};
    queue<TreeNode*> q;
    q.push(root);
    vector<vector<int>>> result;

    while (!q.empty()) {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}

```

Key Tree Templates

```

// Tree height / depth
int height(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}

// Diameter of Binary Tree
int diameter = 0;
int dfs(TreeNode* root) {
    if (!root) return 0;
    int left = dfs(root->left);
    int right = dfs(root->right);
    diameter = max(diameter, left + right);
    return 1 + max(left, right);
}

// Path sum – root to leaf

```

```

bool hasPathSum(TreeNode* root, int target) {
    if (!root) return false;
    if (!root->left && !root->right) return root->val == target;
    return hasPathSum(root->left, target - root->val) ||
        hasPathSum(root->right, target - root->val);
}

// Lowest Common Ancestor
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lca(root->left, p, q);
    TreeNode* right = lca(root->right, p, q);
    if (left && right) return root; // p and q in different subtrees
    return left ? left : right;
}

// Validate BST
bool isValidBST(TreeNode* root, long long min = LLONG_MIN, long long max = LLONG_MAX) {
    if (!root) return true;
    if (root->val <= min || root->val >= max) return false;
    return isValidBST(root->left, min, root->val) &&
        isValidBST(root->right, root->val, max);
}

```

Pattern 8: Graph Patterns

Graph Representations

```

// Adjacency List (most common in interviews)
int n = 5; // nodes
vector<vector<int>>> adj(n); // Unweighted
vector<vector<pair<int,int>>> wadj(n); // Weighted: {neighbor, weight}

// Build from edge list
for (auto& [u, v] : edges) {
    adj[u].push_back(v);
    adj[v].push_back(u); // For undirected
}

// Adjacency Matrix (for dense graphs)
vector<vector<int>>> mat(n, vector<int>(n, 0));
mat[u][v] = 1; // or weight

```

DFS Template

```

vector<bool> visited(n, false);

void dfs(int node, vector<vector<int>>& adj) {
    visited[node] = true;
    // Process node
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj);
        }
    }
}

// Count connected components
int countComponents(int n, vector<vector<int>>& adj) {
    vector<bool> visited(n, false);
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) { dfs(i, adj); count++; }
    }
}

```

```

    }
    return count;
}

```

BFS Template

```

vector<int> bfs(int start, vector<vector<int>>& adj) {
    vector<int> dist(adj.size(), -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (int neighbor : adj[node]) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[node] + 1;
                q.push(neighbor);
            }
        }
    }
    return dist;
}

```

Topological Sort (for DAGs)

```

// Kahn's Algorithm (BFS-based)
vector<int> topoSort(int n, vector<vector<int>>& adj) {
    vector<int> inDegree(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : adj[u]) inDegree[v]++;

    queue<int> q;
    for (int i = 0; i < n; i++) if (inDegree[i] == 0) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--inDegree[v] == 0) q.push(v);
    }
    return order.size() == n ? order : {}; // Empty if cycle exists
}

```

Union-Find (Disjoint Set Union)

```

class UnionFind {
    vector<int> parent, rank;
public:
    UnionFind(int n) : parent(n), rank(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) { // Path compression
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    bool unite(int x, int y) { // Union by rank
        int px = find(x), py = find(y);
        if (px == py) return false; // Already connected
        if (rank[px] < rank[py]) swap(px, py);
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
    }
}

```



```

        return true;
    }

    bool connected(int x, int y) { return find(x) == find(y); }
};

// Number of islands
int numIslands(vector<vector<char>>& grid) {
    int m = grid.size(), n = grid[0].size();
    UnionFind uf(m * n);
    int count = 0;
    int dirs[4][2] = {{0,1},{0,-1},{1,0},{-1,0}};

    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) {
        if (grid[i][j] == '1') {
            count++;
            for (auto& d : dirs) {
                int ni = i+d[0], nj = j+d[1];
                if (ni >= 0 && ni < m && nj >= 0 && nj < n &&
                    if (uf.unite(i*n+j, ni*n+nj)) count--;
            }
        }
    }
    return count;
}

```

Dijkstra's Algorithm

```

vector<int> dijkstra(int src, vector<vector<pair<int,int>>& adj) {
    int n = adj.size();
    vector<int> dist(n, INT_MAX);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // Outdated entry

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

Pattern 9: Dynamic Programming

DP Framework

1. Define subproblem: $dp[i] = ?$
2. Recurrence: $dp[i] = f(dp[i-1], dp[i-2], \dots)$
3. Base cases: $dp[0], dp[1], \dots$
4. Order of computation: bottom-up
5. Answer: $dp[n]$ or $\max(dp)$

Classic DP Problems

```

// 1. Climbing Stairs / Fibonacci
int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) { int c = a+b; a=b; b=c; }
    return b;
}

// 2. House Robber
int rob(vector<int>& nums) {
    int prev2 = 0, prev1 = 0;
    for (int x : nums) {
        int curr = max(prev1, prev2 + x);
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}

// 3. Longest Increasing Subsequence – O(n log n)
int lis(vector<int>& nums) {
    vector<int> tails; // tails[i] = smallest tail of LIS of length i+1
    for (int x : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}

// 4. 0/1 Knapsack
int knapsack(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<int> dp(W+1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = W; w >= weights[i]; w--) { // Right to left!
            dp[w] = max(dp[w], dp[w-weights[i]] + values[i]);
        }
    }
    return dp[W];
}

// 5. Coin Change – Unbounded Knapsack
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1, INT_MAX);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i && dp[i-coin] != INT_MAX)
                dp[i] = min(dp[i], dp[i-coin] + 1);
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}

// 6. Longest Common Subsequence
int lcs(string& s1, string& s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (s1[i-1] == s2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    return dp[m][n];
}

// 7. Edit Distance

```

```
int editDistance(string& s, string& t) {
    int m = s.size(), n = t.size();
    vector<vector<int>>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    return dp[m][n];
}
```

Pattern 10: Backtracking

Concept

Explore all possibilities recursively. Prune branches that can't lead to valid solutions.

Decision Tree:

```

  []
 / \
[1] [2]
 / \
[1,2] [1,3]
```

Templates

```
// General backtracking template
void backtrack(/* state */, /* choices */, vector<result>& results) {
    if (/* goal reached */) {
        results.push_back(/* current state */);
        return;
    }
    for (/* each choice */) {
        if (/* choice is valid */) {
            // Make choice
            backtrack(/* updated state */);
            // Undo choice (backtrack)
        }
    }
}
```

```
// Subsets
vector<vector<int>>> subsets(vector<int>& nums) {
    vector<vector<int>>> result;
    vector<int> curr;
    function<void(int)> bt = [&](int start) {
        result.push_back(curr);
        for (int i = start; i < nums.size(); i++) {
            curr.push_back(nums[i]);
            bt(i+1);
            curr.pop_back();
        }
    };
    bt(0);
    return result;
}
```

```
// Permutations
vector<vector<int>>> permute(vector<int>& nums) {
    vector<vector<int>>> result;
    vector<bool> used(nums.size(), false);
```

```

vector<int> curr;
function<void()> bt = [&]() {
    if (curr.size() == nums.size()) { result.push_back(curr); return; }
    for (int i = 0; i < nums.size(); i++) {
        if (!used[i]) {
            used[i] = true;
            curr.push_back(nums[i]);
            bt();
            curr.pop_back();
            used[i] = false;
        }
    }
};
bt();
return result;
}

// N-Queens
vector<vector<string>>> solveNQueens(int n) {
    vector<vector<string>>> result;
    vector<string> board(n, string(n, '.'));
    set<int> cols, diag1, diag2;

    function<void(int)> bt = [&](int row) {
        if (row == n) { result.push_back(board); return; }
        for (int col = 0; col < n; col++) {
            if (cols.count(col) || diag1.count(row-col) || diag2.count(row+col)) continue;
            cols.insert(col); diag1.insert(row-col); diag2.insert(row+col);
            board[row][col] = 'Q';
            bt(row+1);
            board[row][col] = '.';
            cols.erase(col); diag1.erase(row-col); diag2.erase(row+col);
        }
    };
    bt(0);
    return result;
}

```

Pattern 11: Heap / Priority Queue

Concept

Use heaps when you repeatedly need the minimum/maximum element, especially with a stream of data.

```

// K Closest Points to Origin
vector<vector<int>>> kClosest(vector<vector<int>>& points, int k) {
    // Max-heap of size k (keep k smallest)
    priority_queue<pair<int,int>> pq; // {dist, index}

    for (int i = 0; i < points.size(); i++) {
        int d = points[i][0]*points[i][0] + points[i][1]*points[i][1];
        pq.push({d, i});
        if (pq.size() > k) pq.pop(); // Remove farthest
    }

    vector<vector<int>>> result;
    while (!pq.empty()) {
        result.push_back(points[pq.top().second]);
        pq.pop();
    }
    return result;
}

```

```
// Merge K Sorted Lists
ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b) { return a->val < b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);

    for (ListNode* l : lists) if (l) pq.push(l);

    ListNode dummy(0);
    ListNode* curr = &dummy;
    while (!pq.empty()) {
        curr->next = pq.top(); pq.pop();
        curr = curr->next;
        if (curr->next) pq.push(curr->next);
    }
    return dummy.next;
}

// Find Median from Data Stream
class MedianFinder {
    priority_queue<int> lower; // Max-heap (lower half)
    priority_queue<int, vector<int>, greater<int>> upper; // Min-heap (upper half)
public:
    void addNum(int num) {
        lower.push(num);
        upper.push(lower.top()); lower.pop();
        if (upper.size() > lower.size()) {
            lower.push(upper.top()); upper.pop();
        }
    }
    double findMedian() {
        if (lower.size() > upper.size()) return lower.top();
        return (lower.top() + upper.top()) / 2.0;
    }
};
```

Pattern 12: Intervals

Concept

Sort by start time, then process greedily.

```
// Merge Intervals
vector<vector<int>>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>>> result;

    for (auto& interval : intervals) {
        if (result.empty() || result.back()[1] < interval[0]) {
            result.push_back(interval);
        } else {
            result.back()[1] = max(result.back()[1], interval[1]);
        }
    }
    return result;
}

// Non-overlapping intervals (minimum removals)
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
        return a[1] < b[1]; // Sort by END time (greedy)
    });

    int count = 0, prevEnd = INT_MIN;
    for (auto& interval : intervals) {
```

```

        if (interval[0] >= prevEnd) prevEnd = interval[1];
        else count++; // Overlap – remove this one
    }
    return count;
}

// Meeting rooms II (minimum meeting rooms)
int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<int> starts, ends;
    for (auto& i : intervals) { starts.push_back(i[0]); ends.push_back(i[1]); }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());

    int rooms = 0, maxRooms = 0, endPtr = 0;
    for (int s : starts) {
        if (s < ends[endPtr]) rooms++;
        else endPtr++;
        maxRooms = max(maxRooms, rooms);
    }
    return maxRooms;
}

```

Pattern 13: Greedy

Concept

Make the locally optimal choice at each step. **Proof needed:** Show that greedy choice property holds.

```

// Activity Selection (maximum non-overlapping activities)
int activitySelection(vector<pair<int,int>>& activities) {
    sort(activities.begin(), activities.end(), [](auto& a, auto& b) {
        return a.second < b.second; // Sort by finish time
    });
    int count = 1, lastEnd = activities[0].second;
    for (int i = 1; i < activities.size(); i++) {
        if (activities[i].first >= lastEnd) {
            count++;
            lastEnd = activities[i].second;
        }
    }
    return count;
}

// Jump Game
bool canJump(vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
    }
    return true;
}

// Jump Game II (minimum jumps)
int jump(vector<int>& nums) {
    int jumps = 0, currEnd = 0, farthest = 0;
    for (int i = 0; i < nums.size()-1; i++) {
        farthest = max(farthest, i + nums[i]);
        if (i == currEnd) {
            jumps++;
            currEnd = farthest;
        }
    }
}

```

```

    }
    return jumps;
}

// Gas Station
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.size(); i++) {
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) { start = i+1; tank = 0; }
    }
    return total >= 0 ? start : -1;
}

```

Pattern 14: Trie

Concept

Tree-based data structure for prefix queries. Each path from root to node represents a prefix.

```

struct TrieNode {
    TrieNode* children[26];
    bool isEnd;
    TrieNode() : isEnd(false) {
        fill(children, children+26, nullptr);
    }
};

class Trie {
    TrieNode* root;
public:
    Trie() : root(new TrieNode()) {}

    void insert(string& word) {
        TrieNode* curr = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!curr->children[idx]) curr->children[idx] = new TrieNode();
            curr = curr->children[idx];
        }
        curr->isEnd = true;
    }

    bool search(string& word) {
        TrieNode* curr = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!curr->children[idx]) return false;
            curr = curr->children[idx];
        }
        return curr->isEnd;
    }

    bool startsWith(string& prefix) {
        TrieNode* curr = root;
        for (char c : prefix) {
            int idx = c - 'a';
            if (!curr->children[idx]) return false;
            curr = curr->children[idx];
        }
        return true;
    }
};

```

```
// Word Search II (Trie + DFS)
// Build Trie with all words, then DFS grid pruning with Trie
```

Pattern 15: Bit Manipulation

Core Operations

```
// Check if bit i is set
bool isSet(int n, int i) { return (n >> i) & 1; }

// Set bit i
int setBit(int n, int i) { return n | (1 << i); }

// Clear bit i
int clearBit(int n, int i) { return n & ~(1 << i); }

// Toggle bit i
int toggleBit(int n, int i) { return n ^ (1 << i); }

// Check if power of 2
bool isPow2(int n) { return n > 0 && (n & (n-1)) == 0; }

// Count set bits (Brian Kernighan)
int countBits(int n) {
    int count = 0;
    while (n) { n &= (n-1); count++; } // Remove lowest set bit
    return count;
}

// Find lowest set bit
int lowestBit(int n) { return n & (-n); }

// XOR tricks
// a ^ a = 0      (self-cancel)
// a ^ 0 = a      (identity)
// XOR is commutative and associative

// Find single number (all others appear twice)
int singleNumber(vector<int>& nums) {
    int result = 0;
    for (int x : nums) result ^= x;
    return result;
}

// Missing number
int missingNumber(vector<int>& nums) {
    int n = nums.size(), xorAll = 0, xorNums = 0;
    for (int i = 0; i <= n; i++) xorAll ^= i;
    for (int x : nums) xorNums ^= x;
    return xorAll ^ xorNums;
}

// Generate all subsets using bitmask
vector<vector<int>>> allSubsets(vector<int>& nums) {
    int n = nums.size();
    vector<vector<int>>> result;
    for (int mask = 0; mask < (1 << n); mask++) {
        vector<int> subset;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) subset.push_back(nums[i]);
        result.push_back(subset);
    }
    return result;
}
```



```
// Bit manipulation shortcuts
n << 1 = n * 2      // Left shift = multiply by 2
n >> 1 = n / 2      // Right shift = divide by 2
n & 1 = n % 2       // Check odd/even
~n + 1 = -n         // Two's complement negation
```

Pattern Quick Summary Sheet

Pattern	When to Use	Key Data Structure	Time	Space
Sliding Window	Subarray/substring	deque/map	$O(n)$	$O(k)$
Two Pointers	Sorted array pairs	Indices	$O(n)$	$O(1)$
Binary Search	Sorted/monotonic	Indices	$O(\log n)$	$O(1)$
Prefix Sum	Range sum queries	Array	$O(n)$ precomp	$O(n)$
HashMap	Frequency/lookup	unordered_map	$O(n)$	$O(n)$
Stack	NGE, brackets	stack	$O(n)$	$O(n)$
Tree DFS	Path, subtree	recursion/stack	$O(n)$	$O(h)$
Tree BFS	Level order	queue	$O(n)$	$O(w)$
Graph DFS	Components, path	stack/recursion	$O(V+E)$	$O(V)$
Graph BFS	Shortest path	queue	$O(V+E)$	$O(V)$
DP	Optimal substructure	array/map	$O(n^2)$	$O(n)$
Backtracking	Generate all	recursion	$O(2^n)$	$O(n)$
Heap	K elements, stream	priority_queue	$O(n \log k)$	$O(k)$
Intervals	Overlapping ranges	sorted array	$O(n \log n)$	$O(n)$
Greedy	Optimal local choice	sorted array	$O(n \log n)$	$O(1)$
Trie	Prefix queries	TrieNode	$O(L)$ per op	$O(\text{ALPHABET} \cdot L)$
Bit Manip	XOR, subsets	integers	$O(1)/O(2^n)$	$O(1)$

Next: [Section 5 — Competitive Programming Optimization](#)

Section 5: Competitive Programming Optimization

Fast I/O, Memory & STL Optimization, Advanced Tricks

Goal: Write code that is not just correct, but fast enough to pass tight time limits. These techniques separate good engineers from elite competitive programmers.

Table of Contents

1. Fast I/O
2. Memory Optimization
3. STL Optimization Tricks
4. Recursion Optimization
5. Bitset Tricks
6. Mathematical Optimizations
7. Common Competitive Programming Templates

1. Fast I/O

The Standard Boilerplate

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false); // Decouple C and C++ I/O
    cin.tie(NULL);                    // Untie cin from cout
    cout.tie(NULL);                   // Optional additional speedup

    // Your code here
    return 0;
}
```

Why this works:

- `sync_with_stdio(false)`: By default, C++ streams are synced with C streams (`printf/scanf`). Disabling this makes `cin/cout` much faster.
- `cin.tie(NULL)`: By default, `cin` is tied to `cout` (flushes `cout` before each `cin`). Untying avoids this overhead.
- Warning:** After this, don't mix `cin/cout` with `scanf/printf`.

Speed Comparison

Method	Relative Speed	Use When
<code>scanf/printf</code>	100%	C-style, legacy code

Method	Relative Speed	Use When
<code>cin/cout</code> (default)	~50%	Normal C++
<code>cin/cout</code> (fast I/O)	~95%	Competitive programming
Custom fast read	~200%+	Extreme performance

Custom Fast Input (Ultra-Fast)

```
// Fast integer reader – bypasses stream overhead entirely
inline int readInt() {
    int x = 0; char c = getchar_unlocked();
    while (c < '0' || c > '9') c = getchar_unlocked();
    while (c >= '0' && c <= '9') { x = x*10 + c-'0'; c = getchar_unlocked(); }
    return x;
}

// Fast output
inline void writeInt(int x) {
    if (x == 0) { putchar_unlocked('0'); putchar_unlocked('\n'); return; }
    char buf[20]; int len = 0;
    while (x > 0) { buf[len++] = '0' + x%10; x /= 10; }
    for (int i = len-1; i >= 0; i--) putchar_unlocked(buf[i]);
    putchar_unlocked('\n');
}
```

Output Optimization

```
// SLOW: endl flushes the buffer every time!
for (int i = 0; i < n; i++) cout << result[i] << endl;

// FAST: '\n' doesn't flush
for (int i = 0; i < n; i++) cout << result[i] << '\n';

// Even faster: build string and output once
string out = "";
for (int i = 0; i < n; i++) out += to_string(result[i]) + '\n';
cout << out;

// Or use ostringstream
ostringstream oss;
for (int i = 0; i < n; i++) oss << result[i] << '\n';
cout << oss.str();
```

2. Memory Optimization

Stack vs Heap

```
// Stack – fast, limited (~8MB default)
int arr[1000000]; // 4MB on stack – might cause stack overflow!

// Heap – large, slightly slower
vector<int> arr(1000000); // Safe

// Global arrays – in BSS/data segment, very fast, large limit
int dp[1001][1001]; // 4MB global – SAFE and fast
int grid[1000][1000];
```

```
// Rule: For arrays > ~100KB, use global or heap allocation
```

Memory Pools (for linked structures)

```
// Instead of new/delete for each node (slow allocation):
struct Node {
    int val;
    Node* next;
};

// Memory pool – pre-allocate bulk
const int POOL_SIZE = 100000;
Node pool[POOL_SIZE];
int poolPtr = 0;

Node* newNode(int val) {
    pool[poolPtr].val = val;
    pool[poolPtr].next = nullptr;
    return &pool[poolPtr++];
}

// This is 10-20x faster than individual new
```

Reducing Memory Usage

```
// 1. Use int instead of long long when possible
//    int: 4 bytes, long long: 8 bytes
//    For a dp[1000][1000]: saves 4MB

// 2. Compress coordinates for large ranges
//    Values in [0, 1e9] but only n distinct values
vector<int> vals = /* distinct values */;
sort(vals.begin(), vals.end());
// Compress: original value → compressed index
auto compress = [&](int x) {
    return lower_bound(vals.begin(), vals.end(), x) - vals.begin();
};

// 3. Bitset instead of bool array (8x memory reduction)
bitset<1000000> visited; // 125KB instead of 1MB
visited[i] = 1;
visited.test(i);

// 4. Rolling array (DP space optimization)
// Instead of dp[n][m], use dp[2][m] (only need previous row)
int prev[m+1] = {}, curr[m+1] = {};
// or use only 1D array with careful ordering
```

3. STL Optimization Tricks

Vector Performance

```
// ALWAYS reserve when you know the size
vector<int> v;
v.reserve(n); // Single allocation, then n * O(1) push_back

// emplace_back > push_back for non-trivial types
vector<pair<int,int>> edges;
edges.reserve(m);
edges.emplace_back(u, v); // No temporary object created

// Move semantics for large vectors
vector<int> buildResult() {
```

```

    vector<int> v;
    // ... build v ...
    return v; // RVO (Return Value Optimization) – no copy!
}

// Swap trick to free memory
vector<int> v = {1,2,3,4,5};
v.clear(); // Size = 0, but capacity stays!
vector<int>().swap(v); // Actually free the memory
// Or: v.shrink_to_fit();

```

HashMap Performance

```

// unordered_map with custom hash for better performance
struct CustomHash {
    size_t operator()(int x) const {
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        x = (x >> 16) ^ x;
        return x;
    }
};

unordered_map<int, int, CustomHash> fast_map;

// Avoiding worst case O(n) due to hash collisions
// Standard hash for integers can be hacked in competitive programming
// Solution: use the above custom hash, or use map (O(log n) but reliable)

// Reserve to avoid rehashing
unordered_map<int, int> freq;
freq.reserve(1 << 20); // Power of 2 for efficiency
freq.max_load_factor(0.25); // Lower = fewer collisions, more memory

```

Sort Optimization

```

// Avoid comparison functions for simple types
sort(v.begin(), v.end()); // Faster than custom comparator for ints

// Partial sort – faster than full sort if only need k smallest
partial_sort(v.begin(), v.begin()+k, v.end()); // O(n log k)

// nth_element – O(n) to find kth element without fully sorting
nth_element(v.begin(), v.begin()+k, v.end());
// After: v[k] is the kth smallest, smaller elements before it

// Counting sort for bounded integers
// O(n + range) – much faster than O(n log n) for small ranges
void countSort(vector<int>& arr, int maxVal) {
    vector<int> count(maxVal+1, 0);
    for (int x : arr) count[x]++;
    int idx = 0;
    for (int i = 0; i <= maxVal; i++)
        while (count[i]-- > 0) arr[idx++] = i;
}

// Radix sort for large integer ranges
// O(d * n) where d = number of digits

```

4. Recursion Optimization

Tail Recursion

```
// NOT tail recursive (computation after recursive call)
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n-1); // Multiplication happens AFTER return
}

// Tail recursive (accumulator pattern)
int factorial(int n, int acc = 1) {
    if (n <= 1) return acc;
    return factorial(n-1, n * acc); // Recursive call is last operation
}
// Note: C++ compilers may optimize tail recursion to iteration
```

Memoization vs Bottom-Up DP

```
// Top-down with memoization
// Pros: Only computes needed states, natural recursion structure
// Cons: Recursion overhead, stack limit
unordered_map<int, long long> memo;
long long topDown(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = topDown(n-1) + topDown(n-2);
}

// Bottom-up iterative (preferred for deep recursion)
// Pros: No recursion overhead, predictable memory
// Cons: Must compute all states
long long bottomUp(int n) {
    if (n <= 1) return n;
    vector<long long> dp(n+1);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Stack Overflow Prevention

```
// For deep recursion (n > 10000), convert to iterative
// or increase stack size

// Manual stack to simulate recursion
void dfsIterative(int start, vector<vector<int>&& adj) {
    stack<int> st;
    vector<bool> visited(adj.size(), false);
    st.push(start);

    while (!st.empty()) {
        int node = st.top(); st.pop();
        if (visited[node]) continue;
        visited[node] = true;
        // Process node
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) st.push(neighbor);
        }
    }
}

// For tree DFS with parent tracking
struct Frame {
    int node, parentEdge, childIdx;
};
void treeDP(int root) {
    stack<Frame> st;
    st.push({root, -1, 0});
}
```

```

    while (!st.empty()) {
        auto& [node, par, ci] = st.top();
        if (ci < children[node].size()) {
            int child = children[node][ci++];
            st.push({child, node, 0});
        } else {
            // Post-processing of node
            st.pop();
        }
    }
}

```

5. Bitset Tricks

Basic Bitset Operations

```

#include <bitset>

// Fixed-size bitset (size must be compile-time constant)
bitset<100> b; // 100 bits, all 0
b.set(5);      // Set bit 5
b.reset(5);    // Clear bit 5
b.flip(5);     // Toggle bit 5
b.test(5);     // Check bit 5
b.count();     // Count set bits
b.none();      // True if all 0
b.any();       // True if any 1
b.all();       // True if all 1
b.size();      // Total bits (100)

// Bitwise operations between bitsets
bitset<100> a, b;
a & b; // AND
a | b; // OR
a ^ b; // XOR
~a;    // NOT
a << 1; // Shift left
a >> 1; // Shift right

// Initialize from string or number
bitset<8> bs("10110100");
bitset<8> bn(180); // 180 in binary

```

Bitset for Performance

```

// Problem: Subset sum – is there a subset summing to S?
// O(n * S) standard DP vs O(n * S / 64) with bitset!

bool subsetSum(vector<int>& nums, int S) {
    bitset<10001> dp;
    dp[0] = 1;
    for (int x : nums) dp |= (dp << x);
    return dp[S];
}

// Each bitset operation processes 64 bits at once – 64x speedup!

// Problem: Count set bits in range [l, r]
bitset<1000001> primes;
primes.set();
primes.reset(0); primes.reset(1);
for (int i = 2; i < 1000; i++)
    if (primes.test(i))
        for (int j = i*i; j <= 1000000; j += i) primes.reset(j);

```

```
// Count primes in [l, r]: count bits in range
```

Bitmask DP

```
// Traveling Salesman Problem —  $O(2^n * n^2)$  with bitmask DP
int tsp(int mask, int pos, vector<vector<int>>& dist, vector<vector<int>>& dp) {
    int n = dist.size();
    if (mask == (1 << n) - 1) return dist[pos][0];
    if (dp[mask][pos] != -1) return dp[mask][pos];

    int result = INT_MAX;
    for (int next = 0; next < n; next++) {
        if (mask & (1 << next)) continue;
        int newMask = mask | (1 << next);
        result = min(result, dist[pos][next] + tsp(newMask, next, dist, dp));
    }
    return dp[mask][pos] = result;
}

// Enumerate all subsets of a mask
int mask = 0b1011010;
for (int sub = mask; sub > 0; sub = (sub-1) & mask) {
    // Process sub (subset of mask)
}
// Total iterations:  $3^n$  (sum of  $C(n,k) * 2^k$ )

// Enumerate all submasks of size k
// ... (more complex — see competitive programming resources)
```

6. Mathematical Optimizations

Modular Arithmetic

```
const int MOD = 1e9 + 7;

// All operations modulo MOD
long long add(long long a, long long b) { return (a + b) % MOD; }
long long mul(long long a, long long b) { return a % MOD * (b % MOD) % MOD; }
long long sub(long long a, long long b) { return ((a - b) % MOD + MOD) % MOD; }

// Modular exponentiation:  $a^b \bmod m$ 
long long power(long long a, long long b, long long mod = MOD) {
    long long res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

// Modular inverse (when MOD is prime):  $a^{(MOD-2)} \bmod MOD$ 
long long modInverse(long long a) { return power(a, MOD-2); }

// Modular division
long long divide(long long a, long long b) { return a % MOD * modInverse(b) % MOD; }

// Precompute factorials for combinations
vector<long long> fact(n+1), inv_fact(n+1);
fact[0] = 1;
for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
inv_fact[n] = modInverse(fact[n]);
```



```
for (int i = n-1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD;

// nCr mod p
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] % MOD * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
}
```

GCD and LCM

```
// C++17 built-in
#include <numeric>
int g = gcd(48, 18); // 6
int l = lcm(4, 6);   // 12

// Manual implementation
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a%b); }
long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }

// Extended GCD (for modular inverse when MOD not prime)
long long extGCD(long long a, long long b, long long& x, long long& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long g = extGCD(b, a%b, x1, y1);
    x = y1;
    y = x1 - (a/b) * y1;
    return g;
}
```

Sieve of Eratosthenes

```
// O(n log log n) – find all primes up to n
vector<bool> sieve(int n) {
    vector<bool> isPrime(n+1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; (long long)i*i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i*i; j <= n; j += i) isPrime[j] = false;
        }
    }
    return isPrime;
}

// Linear sieve – O(n) strict
vector<int> linearSieve(int n) {
    vector<int> primes;
    vector<int> minPrime(n+1, 0);
    for (int i = 2; i <= n; i++) {
        if (!minPrime[i]) { minPrime[i] = i; primes.push_back(i); }
        for (int p : primes) {
            if (p > minPrime[i] || (long long)i*p > n) break;
            minPrime[i*p] = p;
        }
    }
    return primes;
}
```

Segment Tree (for range queries)

```
class SegTree {
    int n;
    vector<int> tree;
public:
    SegTree(int n) : n(n), tree(4*n, 0) {}
}
```

```

void build(vector<int>& arr, int node, int start, int end) {
    if (start == end) { tree[node] = arr[start]; return; }
    int mid = (start + end) / 2;
    build(arr, 2*node, start, mid);
    build(arr, 2*node+1, mid+1, end);
    tree[node] = tree[2*node] + tree[2*node+1];
}

void update(int node, int start, int end, int idx, int val) {
    if (start == end) { tree[node] = val; return; }
    int mid = (start + end) / 2;
    if (idx <= mid) update(2*node, start, mid, idx, val);
    else update(2*node+1, mid+1, end, idx, val);
    tree[node] = tree[2*node] + tree[2*node+1];
}

int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return tree[node];
    int mid = (start + end) / 2;
    return query(2*node, start, mid, l, r) +
           query(2*node+1, mid+1, end, l, r);
}

void update(int idx, int val) { update(1, 0, n-1, idx, val); }
int query(int l, int r) { return query(1, 0, n-1, l, r); }
};

```

7. Common Competitive Programming Templates

Complete Competitive Programming Header

```

#include <bits/stdc++.h>
using namespace std;

// Type aliases
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<pii> vpii;
typedef vector<vector<int>> vvi;

// Macros (use sparingly – reduce readability)
#define pb push_back
#define mp make_pair
#define fi first
#define se second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define rep(i,a,b) for(int i=(a);i<(b);i++)
#define FOR(i,n) for(int i=0;i<n;i++)

// Constants
const int INF = 1e9;
const ll LINF = 1e18;
const int MOD = 1e9 + 7;
const double PI = acos(-1.0);
const int dx[] = {0,0,1,-1}; // 4-directional grid
const int dy[] = {1,-1,0,0};

```

```

const int dx8[] = {0,0,1,-1,1,1,-1,-1}; // 8-directional
const int dy8[] = {1,-1,0,0,1,-1,1,-1};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >>> t;
    while (t--) {
        // solve
    }
    return 0;
}

```

Debug Helper

```

// Only active when DEBUG is defined: compile with -DDEBUG
#ifdef DEBUG
    #define dbg(x) cerr <<< #x <<< " = " <<< x <<< "\n"
    #define dbgv(v) { cerr <<< #v <<< " = ["; for (auto x : v) cerr <<< x <<< ",";
n"; }
    #define dbg2d(v) { cerr <<< #v <<< ":\n"; for (auto& row : v) { for (auto x : row) ce
<<< x <<< "\t"; cerr <<< "\n"; } }
#else
    #define dbg(x)
    #define dbgv(v)
    #define dbg2d(v)
#endif

// Usage:
int x = 42;
dbg(x); // In debug mode: x = 42
        // In release mode: no output, no overhead

```

Binary Indexed Tree (Fenwick Tree)

```

class BIT {
    vector<int> tree;
    int n;
public:
    BIT(int n) : n(n), tree(n+1, 0) {}

    void update(int i, int delta) { // O(log n)
        for (i++; i <= n; i += i & (-i)) tree[i] += delta;
    }

    int query(int i) { // Prefix sum [0, i] O(log n)
        int sum = 0;
        for (i++; i > 0; i -= i & (-i)) sum += tree[i];
        return sum;
    }

    int query(int l, int r) { return query(r) - (l > 0 ? query(l-1) : 0); }
};

```

Sparse Table (Range Minimum Query in O(1))

```

class SparseTable {
    vector<vector<int>> table;
    vector<int> log2_;
    int n;
public:
    SparseTable(vector<int>& arr) : n(arr.size()), log2_(arr.size()+1, 0) {

```

```

int LOG = __lg(n) + 1;
table.assign(LOG, vector<int>(n));
table[0] = arr;

for (int i = 2; i <= n; i++) log2_[i] = log2_[i/2] + 1;

for (int j = 1; j < LOG; j++)
    for (int i = 0; i + (1<<j) <= n; i++)
        table[j][i] = min(table[j-1][i], table[j-1][i + (1<<(j-1))]);
}

int query(int l, int r) { // O(1) range minimum
    int k = log2_[r-l+1];
    return min(table[k][l], table[k][r - (1<<k) + 1]);
}
};

```

Performance Tips Summary

Technique	Speedup	Memory Impact	When to Use
Fast I/O	2-4x I/O speed	None	Always in competitive programming
Global arrays	Slight	No stack limit	Large arrays
reserve() for vectors	Avoids reallocations	None	Known size beforehand
emplace_back	Small	None	Complex objects
unordered_map vs map	5-10x avg	Slightly more	No ordered iteration needed
Bitset for DP	64x	Less	Boolean DP on large sets
Bitmask DP	Algorithmic	Exponential	Small n (≤ 20) with subsets
Counting sort	$O(n)$ vs $O(n \log n)$	$O(\text{range})$	Bounded integers
nth_element	$O(n)$ vs $O(n \log n)$	None	Only need kth element
'\n' vs endl	Avoids flush	None	Always in output loops

Next: [Section 6 — Interview Preparation](#)

Section 6: Interview Preparation

FAANG Mindset, Communication Strategy & Problem-Solving Framework

***The secret:** FAANG interviews test not just your code, but how you **think**, **communicate**, and **handle pressure**. This section gives you the mental framework to ace any technical interview.*

Table of Contents

1. FAANG Problem-Solving Mindset
2. The 7-Step Interview Framework
3. Optimal Coding Style
4. Communication Scripts
5. Debugging Strategy
6. Common Traps & How to Avoid Them
7. Company-Specific Insights
8. Pressure Handling
9. Behavioral Interview Framework

1. FAANG Problem-Solving Mindset

The Core Mental Model

Problem → Pattern → Template → Adapt → Code → Verify

Key principles:

1. **Never code immediately.** Think first, code second. An interviewer would rather see you think for 5 minutes than watch you code something wrong for 20 minutes.
2. **Brute force is always valid.** Start by stating the naive solution. This shows you understand the problem. Then optimize.
3. **Talk out loud constantly.** Your interviewer cannot read your mind. Narrate every thought: "I notice this array is sorted, so I'm thinking binary search..."
4. **Ask clarifying questions.** This is not a weakness — it's what senior engineers do. You never code based on incomplete requirements.
5. **Constraints are hints.**
 - $n \leq 20 \rightarrow$ Bitmask/Backtracking
 - $n \leq 1000 \rightarrow O(n^2)$ ok
 - $n \leq 10^6 \rightarrow O(n \log n)$ needed
 - "sorted" $\rightarrow$ Binary search
 - "distinct" or "unique" $\rightarrow$ HashMap/Set

Mental Checklist Before Coding

- [] Did I understand the problem correctly? (Restate it)
- [] What are the input constraints? (n, value ranges, negative numbers?)

```
[ ] What are the output requirements? (Return value? Indices or values?)
[ ] Are there any special cases? (Empty input, single element, all same)
[ ] What is the expected time/space complexity? (Ask if unsure)
[ ] Can the input be modified? (Can I sort it?)
[ ] Do I have a brute force solution? (State it first)
[ ] Can I optimize it? (What's the bottleneck?)
```

2. The 7-Step Interview Framework

Step 1: CLARIFY (2-3 minutes)

You: "Let me make sure I understand the problem..."

- Restate the problem in your own words
- Clarify input format and constraints
- Ask about edge cases explicitly

Example questions to ask:

- "Can the array have negative numbers?"
- "Can we have duplicate elements?"
- "What should we return if there's no solution?"
- "Is the array guaranteed to be non-empty?"
- "Can I assume the input fits in an integer?"
- "Are we optimizing for time or space?"

Step 2: EXAMPLES (2 minutes)

- Walk through the provided example step by step
- Create 2-3 of your own examples
- Include edge cases:
 - * Empty input
 - * Single element
 - * All same elements
 - * Already sorted / reverse sorted
 - * Minimum and maximum values

"Let me trace through the example: input [1,2,3], k=2..."

After step 1: ...

After step 2: ...

Final result: ..."

Step 3: BRUTE FORCE (1-2 minutes)

"The naive approach would be to [describe $O(n^2)$ or worse solution].
This would be $O(n^2)$ time and $O(1)$ space, which is too slow for $n=10^6$.
Let me think about how to optimize..."

Important: Even if you know the optimal solution,
ALWAYS mention brute force first.

Step 4: OPTIMIZE (5-10 minutes)

Optimization checklist:

- [] Is there repeated work? → Memoization/Caching
- [] Is there a sorted structure? → Binary search
- [] Do we need a "current max/min"? → Sliding window or heap
- [] Do we need pair/triplet sums? → Two pointers or hash map
- [] Is the answer monotonic? → Binary search on answer
- [] Do subproblems overlap? → Dynamic programming
- [] Is greedy provably correct here? → Greedy

"I notice that [observation]. This suggests [pattern]."

If I use [data structure], I can reduce [operation] from $O(n)$ to $O(1)$, giving overall $O(n \log n)$ time..."

Step 5: CODE (10-15 minutes)

- Write clean, readable code
- Use meaningful variable names (left/right, not l/r for clarity)
- Add a brief comment for non-obvious sections
- Write helper functions if needed
- Don't delete code – cross it out or comment it

"Let me start coding. I'll write a helper function first...
Here, I'm using a sliding window. The left pointer marks the start..."

Step 6: TEST (3-5 minutes)

- Trace through your code with the example
- Test edge cases you identified in step 2
- Dry-run line by line, tracking variable values

"Let me trace through with input [1,2,3,4,5], k=3:
i=0: window=[1,2,3], sum=6
i=1: window=[2,3,4], sum=9, max=9
..."

Step 7: COMPLEXITY (1-2 minutes)

Always state both time AND space complexity:
"Time complexity: $O(n)$ – single pass through the array
Space complexity: $O(k)$ – the sliding window stores at most k elements

Could we do better? For time complexity, we must read all n elements, so $O(n)$ is optimal. For space, if we can't modify input, $O(k)$ is optimal."

3. Optimal Coding Style

Clean Code Principles for Interviews

```
// GOOD: Descriptive variable names
int longestSubarray(vector<int>& nums, int k) {
    int left = 0, maxLength = 0;
    int currentSum = 0;

    for (int right = 0; right < nums.size(); right++) {
        currentSum += nums[right];

        while (currentSum > k) {
            currentSum -= nums[left];
            left++;
        }

        maxLength = max(maxLength, right - left + 1);
    }
    return maxLength;
}

// BAD: Cryptic names
int f(vector<int>& a, int k) {
    int l=0,r=0,mx=0,s=0;
    for(;r<a.size();r++){s+=a[r];while(s>k)s-=a[l++];mx=max(mx,r-l+1);}
    return mx;
}
```

```
}
```

Style Guidelines

```
// 1. Use meaningful names
int left, right;      // Good (not l, r)
int windowSum;       // Good (not ws)
int maxLength;       // Good (not ans)
TreeNode* current;   // Good (not cur)

// 2. Consistent bracing
if (condition) {
    // body
} else {
    // body
}

// 3. One statement per line
left++;              // Good
left++; right--;    // Acceptable for two related operations

// 4. Extract helper functions
bool isValid(string& s, int left, int right) { /* ... */ }
int getDistance(vector<int&& point1, vector<int&& point2) { /* ... */ }

// 5. Handle edge cases first
vector<int&> solve(vector<int&& nums) {
    if (nums.empty()) return {};
    if (nums.size() == 1) return nums;
    // main logic
}

// 6. Use C++17 features appropriately
for (auto& [key, val] : map) { /* ... */ } // Structured bindings
auto [min_it, max_it] = minmax_element(v.begin(), v.end());
if (auto it = map.find(key); it != map.end()) { /* ... */ }
```

4. Communication Scripts

Opening Scripts

"Let me take a moment to understand the problem..."

"Before I start coding, let me ask a few clarifying questions..."

"I see. So the input is [restate], and we need to return [restate output]. Is that right?"

"Are there any constraints on [time/space/input size] I should know about?"

Thinking Out Loud Scripts

"I'm thinking about this... A naive approach would be..."

"I notice that [observation]. This reminds me of the [pattern name] pattern."

"Let me think about the bottleneck here. Currently, [operation X] takes $O(n^2)$. If I could do that in $O(1)$..."

"What if I sort the array first? That gives me $O(n \log n)$ upfront, but then..."

"I think we can use [data structure] here. Let me think about why..."

"Actually, let me reconsider. I realize [issue with previous approach]..."

"I'm going to try a different approach. Instead of [X], what if we [Y]?"

During Coding Scripts

"I'm initializing a hash map to store [purpose]..."

"The outer loop iterates over [what], and for each [element], we..."

"Here I'm using two pointers. Left starts at 0, right will expand..."

"This line handles the edge case where [condition]..."

"I'll add a comment here because this isn't immediately obvious..."

After Writing Code Scripts

"Let me trace through the example to verify: input = [...]..."

"I think I need to check: what happens when the input is empty?"

"The time complexity is $O(n \log n)$ because [reason]."

"The space complexity is $O(n)$ because we store up to n elements."

"I believe the code is correct. One possible optimization would be..."

"Is there anything you'd like me to clarify or optimize?"

5. Debugging Strategy

When Your Solution Is Wrong

Step 1: Identify the failing case

- "My code gives X but expected Y for input Z"

Step 2: Trace through manually

- Walk through line by line with paper/pen
- Track all variable values
- Don't assume – verify each step

Step 3: Check common issues

- Off-by-one errors ($<$ vs $\leq$, 0-indexed vs 1-indexed)
- Integer overflow (should use long long?)
- Edge cases (empty, single element, all same)
- Loop not starting/ending correctly
- Pointer/reference issues

Step 4: Use print statements (in practice, not interview)

- Add `cout << variable` after suspicious lines

Step 5: Re-read the problem

- Sometimes bugs come from misunderstanding the problem

Common Bug Patterns

```
// 1. Off-by-one
for (int i = 0; i < n; i++)    // OK: iterates 0 to n-1
for (int i = 0; i <= n; i++)   // DANGER: iterates 0 to n, potential OOB
for (int i = 1; i < n; i++)    // OK: starts from 1

// 2. Integer overflow
```

```

int a = 1000000, b = 1000000;
int product = a * b;           // OVERFLOW! Use long long
long long product = (long long)a * b; // CORRECT

// 3. Uninitialized variables
int maxVal;                    // DANGER: garbage value
int maxVal = INT_MIN;          // CORRECT: explicit initialization
int maxVal = nums[0];          // CORRECT: initialize from input

// 4. Empty container access
v.front(); // CRASH if v is empty!
if (!v.empty()) v.front(); // SAFE

// 5. Wrong comparison for floating point
double a = 0.1 + 0.2;
if (a == 0.3) { } // WRONG: floating point precision
if (abs(a - 0.3) < 1e-9) { } // CORRECT

// 6. Modifying container while iterating
for (auto it = v.begin(); it != v.end(); ) {
    if (*it == target) it = v.erase(it); // CORRECT: erase returns next
    else it++;
}

```

6. Common Traps & How to Avoid Them

Trap 1: Integer Overflow

```

// Problem: n up to 10^5, values up to 10^4 → sum can be 10^9 (fits int)
// But: n up to 10^5, values up to 10^9 → sum can be 10^14 (needs long long!)

// Rule: If you multiply two values both up to 10^5+, use long long
long long sum = 0LL; // Initialize with LL
sum += (long long)a * b; // Cast before multiplication

// Check: INT_MAX = 2.1 × 10^9, LLONG_MAX = 9.2 × 10^18

```

Trap 2: Accessing map with [] Creates Default Entry

```

map<string, int> m;
if (m["key"] == 0) { } // BAD: creates "key" with value 0 if not present!

// CORRECT:
if (m.count("key") == 0) { } // Check existence first
if (m.find("key") == m.end()) { } // Use find()

```

Trap 3: Comparators Must Be Strict Weak Ordering

```

// WRONG: can cause undefined behavior!
auto cmp = [](int a, int b) { return a >= b; }; // Not strict!

// CORRECT: strict weak ordering (no equals)
auto cmp = [](int a, int b) { return a > b; }; // Strict greater
auto cmp = [](int a, int b) { return a < b; }; // Strict less

```

Trap 4: Modifying Iterator During Loop

```

// WRONG:
for (auto it = v.begin(); it != v.end(); it++) {
    if (*it == 5) v.erase(it); // Iterator invalidated!
}

```

```
// CORRECT:
v.erase(remove(v.begin(), v.end(), 5), v.end());
```

Trap 5: Signed/Unsigned Comparison

```
int n = -1;
vector<int> v = {1,2,3};
if (n < v.size()) { } // DANGER: v.size() is unsigned, -1 becomes huge!
```

```
// CORRECT:
if (n < (int)v.size()) { }
// Or just use signed types consistently
```

Trap 6: Mid Calculation Overflow

```
int left = 0, right = INT_MAX;
int mid = (left + right) / 2; // OVERFLOW if left + right > INT_MAX!

int mid = left + (right - left) / 2; // CORRECT: always use this!
```

7. Company-Specific Insights

Google

- **Focus:** Algorithmic complexity, clean code
- **Favorite topics:** Graphs, DP, trees, system scalability
- **Style:** Multiple rounds (Coding + System Design + Behavioral)
- **Tips:**
 - Always discuss complexity proactively
 - Code must compile in Docs/Google IDE (practice on Docs)
 - Expect follow-ups to make solution more general/efficient
 - Leadership principles matter: talk about scale

Meta (Facebook)

- **Focus:** Practical problem-solving, code quality
- **Favorite topics:** Arrays, strings, trees, graphs
- **Style:** 2 coding rounds, 1 behavioral (values)
- **Tips:**
 - Write production-quality code with error handling
 - Must know Facebook's core values (move fast, be bold, focus on impact)
 - Behavioral: STAR format, emphasize impact and metrics
 - Often asks: "Is there any other approach you considered?"

Amazon

- **Focus:** Leadership Principles (14 principles)
- **Favorite topics:** Graphs, DP, trees, OOP design
- **Style:** 4-5 rounds (mix of LP + coding)
- **Tips:**
 - Every question has a behavioral component
 - Prepare 2-3 STAR stories for each of the 14 principles

- Customer obsession is most important principle
- Think about scale: "How does this work at Amazon scale?"

Microsoft

- **Focus:** Problem solving + system design
- **Favorite topics:** Trees, graphs, strings, OOP
- **Style:** 4-5 rounds, collaborative
- **Tips:**
 - Interviewers are more collaborative — ask for hints if stuck (they want to help)
 - Focus on growing/learning stories in behavioral
 - System design rounds matter
 - Ask questions about team culture — they appreciate curiosity

Morgan Stanley / Goldman / Citi

- **Focus:** Algorithmic correctness + finance awareness
- **Favorite topics:** Arrays, graphs, dynamic programming
- **Style:** 2-3 coding rounds + risk/trading domain questions
- **Tips:**
 - Know basic financial concepts (options, bonds, risk)
 - Stress-test your solutions (financial systems need high reliability)
 - Discuss error handling and edge cases more than in FAANG
 - Complexity of implementation matters (maintainability > cleverness)

Uber / Airbnb

- **Focus:** Practical implementation, scalability
- **Favorite topics:** Graphs (trip routing!), trees, system design
- **Style:** 2-3 coding + system design
- **Tips:**
 - Uber: Know graph algorithms well (shortest path = routing)
 - Airbnb: Know about search/ranking systems
 - Think about real-world constraints (GPS accuracy, network failures)

8. Pressure Handling

If You're Stuck

1. State what you know: "I understand the problem, and I can see that..."
2. Think out loud about approaches:
"Let me think about what data structures would help here...
A hashmap would give me $O(1)$ lookup, which could help with..."
3. Ask for a hint (it's OK!):
"I'm thinking along the lines of dynamic programming.
Can you give me a hint about the subproblem structure?"
4. Start with brute force:
"Let me at least code the brute force $O(n^2)$ approach,
and then we can optimize from there."
5. Draw examples:

"Let me draw out what should happen with this input..."

If You Make a Mistake

"Actually, I realize that's wrong because [reason].
Let me fix that..."

"I see the issue – I forgot to handle the case where [edge case].
Let me add that..."

Never say: "Oh that won't work" and delete everything.
Instead: "That approach has a flaw. Let me adjust..."

Time Management

1-2 min: Clarify problem
2-3 min: Work through examples
2-3 min: Discuss brute force + optimization
10-15 min: Code the solution
3-5 min: Test and debug
2 min: Analyze complexity
2 min: Questions for interviewer

9. Behavioral Interview Framework

STAR Method

S - Situation: Set the context (1-2 sentences)
T - Task: What was your responsibility? (1 sentence)
A - Action: What did YOU specifically do? (3-5 sentences, most important)
R - Result: What was the outcome? (quantify if possible)

Prepare These Stories (with C++/Engineering context)

1. Most challenging technical problem you've solved
→ Focus on algorithm design, debugging, complexity
2. Time you disagreed with your team / handled conflict
→ Show technical diplomacy: "I presented benchmarks showing..."
3. Time you had to learn something quickly
→ Learning C++ for performance-critical system
4. Biggest failure and what you learned
→ Production bug, chose wrong algorithm, fix + learnings
5. Time you improved a process / mentored someone
→ Code review improvements, DSA study group
6. Most impactful project
→ Quantify: reduced latency by X%, saved \$Y, handled Z requests/sec

Questions to Ask Interviewers

Technical:

- "What are the biggest technical challenges your team faces?"
- "How does the team handle technical debt?"
- "What does the tech stack look like? Any migration happening?"

Growth:

- "What does success look like in the first 6 months?"
- "How do engineers grow from senior to staff level here?"
- "What learning opportunities exist?"

Culture:

- "How does the team handle failure?"
 - "What's the on-call culture like?"
 - "What's one thing you wish you'd known before joining?"
-

Pre-Interview Checklist

One Week Before

- [] Review all 15 patterns – can you recognize and code from memory?
- [] Practice 2-3 hard problems per day
- [] Do 2 mock interviews (use Pramp, interviewing.io, or with a peer)
- [] Review your top 5 behavioral stories
- [] Research the company (recent news, products, engineering blog)

One Day Before

- [] Light review – don't learn new material
- [] Review your cheatsheets (STL, patterns)
- [] Sleep 7-8 hours
- [] Prepare your setup: extra monitors, headphones, water
- [] Know the interview format and who you're interviewing with

Interview Day

- [] Test audio, video, coding environment 30 minutes early
 - [] Have a pen and paper for diagrams
 - [] Keep your STL cheatsheet visible (for offline interviews)
 - [] Breathe – you've prepared well
-

Next: Section 7 — 1 Month Roadmap

Section 7: 1-Month FAANG Preparation Roadmap

Structured Daily Plan for C++ DSA Mastery

*****Philosophy:**** Consistency beats intensity. 4-5 focused hours daily for 30 days is better than 12-hour cramming sessions. With your 5 years of backend experience, you have a huge advantage — you understand systems, can estimate complexity intuitively, and write clean code.*

Overview

Week	Theme	Goal
Week 1	C++ + Foundations + Basic Patterns	Comfortable with C++ and core patterns
Week 2	Core DSA Patterns	Master sliding window, two pointers, hashing, sorting
Week 3	Advanced Patterns	Trees, graphs, DP, backtracking
Week 4	Mock Interviews + Company Prep	Interview-ready, pattern fluency

Daily Structure (4-5 Hours)

- Morning (1.5-2 hrs): Learn/Review
- Study theory from this handbook
 - Review pattern template
 - Understand 1-2 new concepts
- Afternoon (2-2.5 hrs): Practice
- Solve 3-5 LeetCode problems (Easy → Medium → Hard)
 - Time yourself (45 min per problem max)
 - Review solutions after
- Evening (30 min): Revision
- Review what you learned today
 - Update your personal cheatsheet
 - Quick mock communication practice

Week 1: C++ Mastery + DSA Foundations

Day 1 — C++ Syntax & Types

- Theory (from Section 1):**
- Variables, data types, operators
 - Conditions, loops
 - Functions (by value vs by reference)
 - Type casting, overflow
- LeetCode Problems:**

1. Two Sum — Easy
2. Palindrome Number — Easy
3. Reverse Integer — Medium
4. FizzBuzz — Easy

C++ Focus: Practice declaring variables, writing loops, using cin/cout

Day 2 — Arrays & Strings

Theory:

- vector vs array
- String operations (substr, find, compare)
- STL: sort, reverse, find, count

LeetCode Problems:

1. Best Time to Buy and Sell Stock — Easy
 2. Contains Duplicate — Easy
 3. Valid Anagram — Easy
 4. Maximum Subarray — Medium (Kadane's)
-

Day 3 — HashMap & HashSet

Theory (Section 2: map/unordered<map/set/unordered<set>):

- When to use each
- $O(1)$ vs $O(\log n)$ tradeoffs
- Custom hash, collision handling

LeetCode Problems:

1. Two Sum — Revisit with hashmap
 2. Group Anagrams — Medium
 3. Top K Frequent Elements — Medium
 4. Longest Consecutive Sequence — Medium
-

Day 4 — Binary Search Fundamentals

Theory (Section 3, Section 4 Pattern 3):

- Classic binary search
- Lower/upper bound
- Binary search on answer pattern

LeetCode Problems:

1. Binary Search — Easy
 2. Search Insert Position — Easy
 3. First Bad Version — Easy
 4. Koko Eating Bananas — Medium
 5. Search in Rotated Sorted Array — Medium
-

Day 5 — Recursion & Backtracking Introduction

Theory (Section 3: Recursion, Section 4 Pattern 10):

- Recursion tree visualization
- Base case + recursive case
- Subsets and permutations

LeetCode Problems:

1. Fibonacci Number — Easy
 2. Climbing Stairs — Easy
 3. Subsets — Medium
 4. Permutations — Medium
-

Day 6 — Sorting Algorithms

Theory (Section 3):

- Merge sort implementation
- Quick sort implementation
- When to use which sort
- Custom comparators

LeetCode Problems:

1. Sort Colors — Medium (Dutch flag)
 2. Merge Intervals — Medium
 3. Largest Number — Medium
 4. Kth Largest Element — Medium
-

Day 7 — Week 1 Review & Mock

Morning: Review all week 1 material. Fill gaps.

Practice: Solve without looking at solutions:

1. Two Sum II — Medium
2. 3Sum — Medium
3. Find Minimum in Rotated Sorted Array — Medium

Mock Interview: Time yourself (45 min/problem). Speak out loud.

Week 2: Core Patterns Mastery

Day 8 — Sliding Window

Theory (Section 4 Pattern 1):

- Fixed-size window
- Variable-size window
- Monotonic deque

LeetCode Problems:

1. Maximum Average Subarray I — Easy
 2. Longest Substring Without Repeating Characters — Medium
 3. Minimum Window Substring — Hard
 4. Sliding Window Maximum — Hard
-

Day 9 — Two Pointers

Theory (Section 4 Pattern 2):

- Opposite direction
- Same direction (fast/slow)
- Floyd's cycle detection

LeetCode Problems:

1. Valid Palindrome — Easy
 2. Container With Most Water — Medium
 3. 3Sum — Medium (revisit)
 4. Trapping Rain Water — Hard
 5. Linked List Cycle — Easy
-

Day 10 — Prefix Sum

Theory (Section 4 Pattern 4):

- 1D and 2D prefix sums
- Subarray sum with hashmap trick

LeetCode Problems:

1. Range Sum Query — Easy
 2. Subarray Sum Equals K — Medium
 3. Product of Array Except Self — Medium
 4. Count Number of Nice Subarrays — Medium
-

Day 11 — Stack & Monotonic Stack

Theory (Section 4 Pattern 6):

- Stack for bracket matching
- Monotonic stack for NGE
- Histogram rectangle

LeetCode Problems:

1. Valid Parentheses — Easy
 2. Min Stack — Medium
 3. Daily Temperatures — Medium
 4. Largest Rectangle in Histogram — Hard
 5. Decode String — Medium
-

Day 12 — Linked Lists

Theory:

- ListNode structure
- Reversal, cycle detection, merge
- Fast/slow pointers

LeetCode Problems:

1. Reverse Linked List — Easy
 2. Merge Two Sorted Lists — Easy
 3. Linked List Cycle II — Medium
 4. Remove Nth Node From End — Medium
 5. LRU Cache — Medium
-

Day 13 — Heap/Priority Queue

Theory (Section 4 Pattern 11):

- Max-heap, min-heap
- K-th element problems
- Stream processing

LeetCode Problems:

1. Kth Largest Element — Medium
 2. Top K Frequent Elements — Medium
 3. K Closest Points to Origin — Medium
 4. Find Median from Data Stream — Hard
 5. Merge K Sorted Lists — Hard
-

Day 14 — Week 2 Review & Mock

Full Mock Interview (2 problems, 45 min each):

Randomly pick from:

- Sliding window
- Two pointers
- HashMap
- Stack

Reflection:

- Which patterns are weak? → More focus in week 3
 - Timing: Are you finishing in 45 min?
 - Communication: Are you thinking out loud?
-

Week 3: Advanced Patterns

Day 15 — Binary Trees

Theory (Section 4 Pattern 7):

- DFS: inorder, preorder, postorder
- BFS: level order

- TreeNode struct, recursion patterns

LeetCode Problems:

1. <i><u>Maximum Depth of Binary Tree</u></i> — Easy
 2. <i><u>Invert Binary Tree</u></i> — Easy
 3. <i><u>Diameter of Binary Tree</u></i> — Easy
 4. <i><u>Binary Tree Level Order Traversal</u></i> — Medium
 5. <i><u>Lowest Common Ancestor of BST</u></i> — Medium
-

Day 16 — Binary Tree Advanced + BST

LeetCode Problems:

1. <i><u>Validate Binary Search Tree</u></i> — Medium
 2. <i><u>Binary Tree Right Side View</u></i> — Medium
 3. <i><u>Construct Binary Tree from Preorder and Inorder</u></i> — Medium
 4. <i><u>Serialize and Deserialize Binary Tree</u></i> — Hard
 5. <i><u>Binary Tree Maximum Path Sum</u></i> — Hard
-

Day 17 — Graph Fundamentals

Theory (Section 4 Pattern 8):

- Adjacency list/matrix
- DFS and BFS templates
- Connected components

LeetCode Problems:

1. <i><u>Number of Islands</u></i> — Medium
 2. <i><u>Clone Graph</u></i> — Medium
 3. <i><u>Course Schedule</u></i> — Medium (Topological sort)
 4. <i><u>Pacific Atlantic Water Flow</u></i> — Medium
 5. <i><u>Word Ladder</u></i> — Hard (BFS)
-

Day 18 — Graph Advanced (Dijkstra, Union-Find)

Theory:

- Dijkstra's shortest path
- Union-Find / Disjoint Set Union
- Minimum spanning tree (Kruskal/Prim)

LeetCode Problems:

1. <i><u>Network Delay Time</u></i> — Medium (Dijkstra)
 2. <i><u>Number of Provinces</u></i> — Medium (Union-Find)
 3. <i><u>Redundant Connection</u></i> — Medium (Union-Find)
 4. <i><u>Min Cost to Connect All Points</u></i> — Medium (Prim/Kruskal)
 5. <i><u>Cheapest Flights Within K Stops</u></i> — Medium
-

Day 19 — Dynamic Programming I (1D)

Theory (Section 4 Pattern 9):

- DP framework: subproblem, recurrence, base cases
- Linear DP: climbing stairs, house robber
- LIS with binary search

LeetCode Problems:

1. <i><u>Climbing Stairs</u></i> — Easy
 2. <i><u>House Robber</u></i> — Medium
 3. <i><u>House Robber II</u></i> — Medium
 4. <i><u>Longest Increasing Subsequence</u></i> — Medium
 5. <i><u>Word Break</u></i> — Medium
-

Day 20 — Dynamic Programming II (2D)

Theory:

- Grid DP
- String DP (LCS, Edit Distance)
- 0/1 Knapsack

LeetCode Problems:

1. <i><u>Unique Paths</u></i> — Medium
 2. <i><u>Longest Common Subsequence</u></i> — Medium
 3. <i><u>Edit Distance</u></i> — Hard
 4. <i><u>Coin Change</u></i> — Medium
 5. <i><u>Partition Equal Subset Sum</u></i> — Medium
-

Day 21 — Backtracking + Intervals + Greedy

Theory (Section 4 Patterns 10, 12, 13):

LeetCode Problems:

1. <i><u>Combination Sum</u></i> — Medium
 2. <i><u>N-Queens</u></i> — Hard
 3. <i><u>Merge Intervals</u></i> — Medium (revisit)
 4. <i><u>Non-overlapping Intervals</u></i> — Medium
 5. <i><u>Jump Game II</u></i> — Medium
-

Week 4: Interview Simulation & Company Prep

Day 22 — Trie + Bit Manipulation

Theory (Section 4 Patterns 14, 15):

LeetCode Problems:

1. <i><u>Implement Trie</u></i> — Medium
2. <i><u>Word Search II</u></i> — Hard
3. <i><u>Single Number</u></i> — Easy

4. [Number of 1 Bits](#) — Easy
 5. [Missing Number](#) — Easy
-

Day 23 — Company-Specific: Google/Meta

Google-style (algorithmic, complex):

1. [Median of Two Sorted Arrays](#) — Hard
2. [Regular Expression Matching](#) — Hard
3. [Trapping Rain Water](#) — Hard
4. [Alien Dictionary](#) — Hard

Meta-style (practical, clean code):

1. [Flatten Binary Tree to Linked List](#) — Medium
 2. [Remove Invalid Parentheses](#) — Hard
-

Day 24 — Company-Specific: Amazon/Microsoft

Amazon (LP + algorithm):

1. [LRU Cache](#) — Medium
 2. [Find All Anagrams in a String](#) — Medium
 3. [Copy List with Random Pointer](#) — Medium
 4. [Design Add and Search Words](#) — Medium
-

Day 25 — Company-Specific: Uber/Airbnb/Banks

Focus: Graphs + System thinking:

1. [Course Schedule II](#) — Medium
 2. [Reconstruct Itinerary](#) — Hard
 3. [Task Scheduler](#) — Medium
 4. [Max Points on a Line](#) — Hard
-

Day 26-27 — Full Mock Interviews

Day 26 Mock 1:

- Set timer for 45 minutes
- Pick 1 random medium problem
- Speak aloud, follow the 7-step framework
- Review and grade yourself

Day 27 Mock 2:

- Use [interviewing.io](#) or [Pramp](#)
 - Real interview with another engineer
 - Focus on communication quality
-

Day 28 — Hard Problems Sprint

Challenge mode — solve all in 1 hour each:

1. Word Search II — Hard
2. Minimum Window Substring — Hard
3. Maximum Profit in Job Scheduling — Hard

Day 29 — Weak Areas + System Design Review

Morning: Identify 3 patterns you're weakest at. Solve 2 problems each.

Afternoon: Review system design concepts relevant to DSA:

- Consistent hashing → Hashmap internals
- B-Trees → BST/self-balancing trees
- Skip lists → Balanced BST alternative
- Bloom filters → Probabilistic data structures

Day 30 — Final Review & Mental Preparation

Morning:

- Review your personal cheatsheet
- Go through pattern summary table (Section 4)
- Review STL cheatsheet (Section 2)

Afternoon:

- Solve 3-5 easy problems to build confidence
- Review behavioral stories
- Prepare questions to ask interviewers

Evening:

- Rest
- Visualize yourself succeeding in the interview
- Sleep 8 hours

LeetCode Top 75 Roadmap (Categorized)

Arrays & Hashing

#	Problem	Difficulty	Pattern
1	Two Sum	Easy	HashMap
2	Contains Duplicate	Easy	HashSet
3	Valid Anagram	Easy	HashMap
4	Group Anagrams	Medium	HashMap
5	Top K Frequent	Medium	HashMap + Heap
6	Encode/Decode Strings	Medium	String
7	Product of Array Except Self	Medium	Prefix/Suffix
8	Valid Sudoku	Medium	HashSet

#	Problem	Difficulty	Pattern
9	Longest Consecutive Sequence	Medium	HashSet

Two Pointers

#	Problem	Difficulty	Pattern
10	Valid Palindrome	Easy	Two Pointers
11	Two Sum II	Medium	Two Pointers
12	3Sum	Medium	Sort + Two Pointers
13	Container With Most Water	Medium	Two Pointers
14	Trapping Rain Water	Hard	Two Pointers

Sliding Window

#	Problem	Difficulty	Pattern
15	Best Time to Buy/Sell Stock	Easy	Sliding Window
16	Longest Substring Without Repeating	Medium	SW Variable
17	Longest Repeating Char Replacement	Medium	SW + Counter
18	Permutation In String	Medium	SW Fixed
19	Minimum Window Substring	Hard	SW Variable
20	Sliding Window Maximum	Hard	SW + Deque

Binary Search

#	Problem	Difficulty	Pattern
21	Binary Search	Easy	Classic BS
22	Search a 2D Matrix	Medium	BS
23	Koko Eating Bananas	Medium	BS on Answer
24	Find Min in Rotated Sorted Array	Medium	BS
25	Search in Rotated Sorted Array	Medium	BS
26	Time Based Key-Value Store	Medium	BS
27	Median of Two Sorted Arrays	Hard	BS

Stack

#	Problem	Difficulty	Pattern
28	Valid Parentheses	Easy	Stack
29	Min Stack	Medium	Stack
30	Evaluate Reverse Polish Notation	Medium	Stack
31	Generate Parentheses	Medium	Backtrack
32	Daily Temperatures	Medium	Monotonic Stack
33	Car Fleet	Medium	Monotonic Stack
34	Largest Rectangle in Histogram	Hard	Monotonic Stack

Linked List

#	Problem	Difficulty	Pattern
35	Reverse Linked List	Easy	Two Pointer
36	Merge Two Sorted Lists	Easy	Merge
37	Reorder List	Medium	Fast/Slow
38	Remove Nth From End	Medium	Fast/Slow
39	Linked List Cycle	Easy	Fast/Slow
40	Find Duplicate Number	Medium	Floyd's Cycle
41	LRU Cache	Medium	HashMap + DLL
42	Merge K Sorted Lists	Hard	Heap
43	Reverse Nodes in K-Group	Hard	Recursion

Trees

#	Problem	Difficulty	Pattern
44	Invert Binary Tree	Easy	DFS
45	Maximum Depth	Easy	DFS
46	Diameter of Tree	Easy	DFS
47	Balanced Binary Tree	Easy	DFS
48	Same Tree	Easy	DFS
49	Subtree of Another Tree	Easy	DFS
50	LCA of BST	Medium	DFS
51	BST Insert/Delete	Medium	BST
52	Validate BST	Medium	DFS
53	Kth Smallest in BST	Medium	Inorder
54	Construct BST from Preorder	Medium	DFS
55	Level Order Traversal	Medium	BFS

#	Problem	Difficulty	Pattern
56	Right Side View	Medium	BFS
57	Count Good Nodes	Medium	DFS
58	Word Search II	Hard	Trie + DFS
59	Max Path Sum	Hard	DFS
60	Serialize/Deserialize Tree	Hard	BFS/DFS

Graphs

#	Problem	Difficulty	Pattern
61	Number of Islands	Medium	DFS/BFS
62	Clone Graph	Medium	DFS/BFS
63	Max Area of Island	Medium	DFS
64	Pacific Atlantic Water Flow	Medium	DFS
65	Surrounded Regions	Medium	DFS
66	Rotting Oranges	Medium	BFS
67	Walls and Gates	Medium	BFS
68	Course Schedule	Medium	Topo Sort
69	Course Schedule II	Medium	Topo Sort
70	Redundant Connection	Medium	Union-Find
71	Word Ladder	Hard	BFS

DP

#	Problem	Difficulty	Pattern
72	Climbing Stairs	Easy	1D DP
73	House Robber	Medium	1D DP
74	Coin Change	Medium	Unbounded Knapsack
75	Longest Increasing Subsequence	Medium	DP + BS

Revision Schedule

Daily Quick Revision (15 min/day)

Monday: Sliding Window + Two Pointers
 Tuesday: Binary Search + Prefix Sum
 Wednesday: Stack + HashMap
 Thursday: Trees (DFS/BFS templates)
 Friday: Graphs + DP templates
 Saturday: Mock interview + Backtracking
 Sunday: Rest or catch-up

Pattern Template Flashcard System

Create flashcards (physical or Anki) for:

1. Sliding window variable template
2. Binary search on answer template
3. DFS tree template
4. BFS graph template
5. DP coin change template
6. Union-Find template
7. Monotonic stack NGE template
8. Backtracking subsets template
9. Dijkstra's template
10. Prefix sum + hashmap template

Review all 10 flashcards daily (5-10 min).

Mock Interview Schedule

Day	Type	Platform
7	Solo mock	Timer + speak aloud
14	Solo mock	Timer + speak aloud
21	Peer mock	With a friend
24	Professional mock	interviewing.io
26	Solo mock	Full simulation
27	Peer/Professional mock	Pramp
29	Final solo	Confidence builder

Progress Tracker

Weekly Pattern Mastery Self-Assessment

Rate yourself 1-5 for each pattern after each week:

Pattern	Week 1	Week 2	Week 3	Week 4
Sliding Window				
Two Pointers				
Binary Search				
Prefix Sum				
HashMap				
Stack				
Tree DFS				
Tree BFS				
Graph DFS/BFS				

Pattern	Week 1	Week 2	Week 3	Week 4
Dynamic Programming				
Backtracking				
Heap				
Intervals				
Greedy				
Trie				
Bit Manipulation				

Score 1: Can't recognize the pattern

Score 2: Can recognize but not code

Score 3: Can code with hints/reference

Score 4: Can code independently in 30 min

Score 5: Can solve optimally + explain + handle follow-ups in < 20 min

Target: All patterns at 4+ before interviews.

Resources

Primary (This Handbook)

- Section 1-7 as your core reference

LeetCode

- [NeetCode 150](#) — Best curated list
- [Blind 75](#) — Classic list
- Use LeetCode Premium for company-specific questions

For Deeper Learning

- **Book:** "Introduction to Algorithms" (CLRS) — Reference for theory
- **Book:** "Competitive Programming 3" — Advanced techniques
- **YouTube:**
- NeetCode (pattern explanations)
- Back To Back SWE (deep algorithmic thinking)
- Abdul Bari (algorithms foundation)

Practice Platforms

- [LeetCode](#) — Primary
- [HackerRank](#) — Good for C++ syntax
- [Codeforces](#) — Competitive programming
- [AlgoExpert](#) — Video solutions

